Laporan Tugas Kecil

Strategi Algoritma (IF2211)

Disusun Oleh

Kelompok:

Raysha Erviandika Putra (13524050)

Muhammad Daffa Arrizki Yanma (13524133)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG 2026

BAB I

ALGORITMA PATHFINDING

1. Algoritma

a. Overview

Program ini menyelesaikan permasalahan *ice sliding puzzle* dengan tiga algoritma pathfinding utama yaitu Uniform Cost Search (UCS), Greedy Best First Search (GBFS), dan juga A*. Ketiga algoritma ini bekerja secara modular pada struktur representasi masalah yang sama yaitu dengan *board* dan *tiles* yang dibedakan antara cost dan juga tipe tile.

Algoritma memiliki alur umum sebagai berikut:

1. File config .txt yang memiliki data kondisi awal board akan di inisialisasi oleh Parser.java untuk membentuk objek dari kelas board baru, proses ini diikuti dengan validasi.
2. Board adalah data type yang menyimpan seluruh kondisi tile, dalam hal ini board menyimpan cost dari tiap tile, tile start, tile goal, tile dengan *order* dimana mulai saat ini *ordered* tile adalah tile yang harus dilewati pemain jika ada, secara berurutan. Dalam hal ini tila dan board direpresentasikan dengan struktur graf.
3. Fungsi Solve(Board) yang dimiliki setiap kelas algoritma akan melakukan solve sesuai dengan algoritma yang dimiliki nya dan mengembalikan result yang berisikan solusi gerakan setiap iterasi.
4. Setiap iterasi, algoritma akan membuat salinan dari kondisi board saat ini dan menyimpannya. Pada setiap iterasi algoritma akan mencoba bergerak ke 4 arah UP (U), DOWN (D), LEFT (L), RIGHT (R) menggunakan fungsi tileIfMove() yang melakukan perhitungan *sliding* dari player dengan iterasi setiap tile yang dilewati.
5. Jika gerakan valid, cost gerakan dihitung oleh calcCost(...), path diperbarui, lalu state baru dimasukkan ke PriorityQueue
6. State dianggap solusi jika posisi sudah berada di goal dan semua angka wajib sudah dilalui sesuai urutan.

a. States

Posisi “player” tidak cukup direpresentasikan dengan row dan col posisi saat ini karena state yang dimiliki suatu instance dari iterasi bisa saja berbeda dengan iterasi lain. Apabila kondisi board dan juga status passed tile disimpan secara global dapat menyebabkan suatu iterasi setelahnya akan melanggar aturan ordered tiles karena sudah dilewati iterasi sebelumnya. Oleh karena itu state yang dipakai adalah:

```
public record StateKey(int row, int col,
int lastPassedOrder) {}
public StateKey stateKey(Tile currentTile) {
   return new StateKey(currentTile.row, currentTile.col,
lastPassedOrder());
```

```
    }
```

Di dalam algoritma, state key digunakan untuk menyaring state yang sudah pernah dieksplorasi dengan proses adalah sebagai berikut:

1. Ketika sebuah node diambil dari PriorityQueue, algoritma membentuk stateKey dari node tersebut,
2. Jika key itu sudah ada di visited, node dilewati karena dianggap sudah pernah dianalisis,
3. Jika belum ada, key dimasukkan ke visited lalu node diekspansi.

b. Movement

Karena pergerakan bukan hanya 1 tile, tetapi *sliding* sampai tidak dapat bergerak atau pergerakan tidak valid.

```
    public Tile tileIfMove(Tile startTile, Direction dir)
{
        Tile temp = startTile;

        while (temp != null) {
            Tile nextTile = nextTile(temp, dir);

            if (nextTile == null) {
                return null;
            }
            if (nextTile.isWall()) {
                break;
            }

            if (nextTile.isLava()) {
                return null;
            }
            if (!nextTile.canEnter()) {
                return null;
            }
            if (nextTile.order != -1) {
                nextTile.hasBeenPassed = true;
            }
            temp = nextTile;
        }
        return temp;
```

```
    }
```

Fungsi tersebut menangani aturan inti permainan yaitu jika arah gerak keluar papan, state dianggap gagal, jika melewati lava, state tidak valid, jika masuk ke tile angka sebelum angka sebelumnya dilewati, state tidak valid, jika gerakan valid, karakter terus meluncur sampai tepat berhenti sebelum dinding. `nextTile.canEnter()` digunakan untuk melakukan pengecekan apabila tile yang dilewati valid (hal ini termasuk ordered tile), dan `NextTile.hasBeenPassed = true` digunakan untuk menandakan tile tersebut sudah dilewati jika merupakan ordered tile.

```
public int calcCost(Tile startTile, Tile endTile,
Direction dir) {
        int cost = 0;
        Tile temp = startTile;

        while (temp != endTile) {
            temp = nextTile(temp, dir);
            cost += temp.cost;
        }
        return cost;
    }
```

`calcCost` digunakan untuk menghitung cost path yang dilewati, sistem pergerakannya sama dengan tileIfMove() hanya saja tidak melakukan pengecekan tile valid.

2. **Heuristic Search**

   Heuristic search adalah pendekatan pencarian yang menggunakan nilai heuristik untuk menilai suatu simpul. Heuristik digunakan untuk memperkirakan nilai atau potensi suatu simpul, misalnya seberapa menjanjikan simpul tersebut, seberapa sulit subpersoalan yang tersisa, kualitas solusi yang diwakili oleh simpul, atau banyaknya informasi yang diperoleh dari simpul tersebut.

3. **UCS**

   UCS adalah algoritma pencarian yang memilih simpul untuk diekspansi berdasarkan biaya jalur jalur terkecil dari simpul awal. Berbeda dengan DFS, BFS, dan IDS, UCS melihat total biaya perjalanan, karena jumlah langkah yang lebih sedikit belum tentu menghasilkan jalur dengan biaya yang paling rendah.

   Pada UCS, setiap simpul disimpan bersama nilai biaya jalurnya. Simpul dengan nilai *g(n)* terkecil yang akan dipilih untuk diekspansi.

   Untuk implementasi pada tugas ini UCS digunakan untuk mencari jalur dari titik awal Z menuju goal O dengan mempertimbangkan total biaya lintasan. Setiap tile memiliki cost

masing masing, sehingga jalur terpendek secara jumlah langkah belum tentu menjadi jalur terbaik untuk dilewati. UCS menyelesaikan masalah tersebut dengan selalu memilih state yang memiliki total biaya paling kecil dari start sampai posisi *n*.

```
private class Node implements Comparable<Node> {
    Tile tile;
    List<Board.Direction> path;
    int cost;
    Board BoardCon;
}
```

tile menyimpan posisi agen saat ini, path menyimpan daftar gerakan yang diambil, cost menyimpan total biaya dari start, dan BoardCon menyimpan kondisi board pada state tersebut

```
@Override
public int compareTo(Node other) {
    return Integer.compare(this.cost, other.cost);
}
```

Dengan compareTo ini, PriorityQueue selalu dapat mengeluarkan node dengan cost terkecil.

```
queue.add(new Node(startBoard.start, new ArrayList<>(),
0, startBoard));

while (!queue.isEmpty()) {
    Node current = queue.poll();

    if (current.BoardCon.isGoalReached(current.tile)) {
        return finishRun(new Result(true, current.path,
current.cost, iterations), startTime);
    }

    for (Board.Direction dir : Board.Direction.values())
{
        Board nextBoard = current.BoardCon.copy();
        Tile nextStart =
nextBoard.getTile(current.tile.row, current.tile.col);
        Tile target = nextBoard.tileIfMove(nextStart,
```

```
dir);

        if (target != null) {
            int moveCost = nextBoard.calcCost(nextStart,
target, dir);
            List<Board.Direction> nextPath = new
ArrayList<>(current.path);
            nextPath.add(dir);

            queue.add(new Node(target, nextPath,
current.cost + moveCost, nextBoard));
        }
    }
}
```

Alur Pathfinding UCS sebagai berikut:

1. Board awal disalin menggunakan board.copy().
2. Node start dimasukkan ke queue dengan path kosong dan cost 0.
3. Selama queue belum kosong, program mengambil node dengan cost terkecil.
4. Program mengecek apakah node tersebut sudah mencapai goal menggunakan isGoalReached.
5. Jika belum, program mencoba semua arah gerak: UP, DOWN, LEFT, dan RIGHT.
6. Untuk setiap arah, program menghitung posisi akhir gerakan dengan tileIfMove.
7. Jika gerakan valid, program menghitung biaya gerakan dengan calcCost.
8. Path baru dibuat dengan menyalin path lama lalu menambahkan arah gerakan terbaru.
9. Node baru dimasukkan lagi ke priority queue dengan total cost yang sudah diperbarui.

Pada program ini, satu aksi bergerak terus ke satu arah sampai berhenti atau *sliding* sampai tidak dapat bergerak atau pergerakan tidak valid. Aturan ini diatur oleh tileIfMove dan biaya gerakan dihitung oleh calcCost

4. GBFS

GBFS adalah algoritma pencarian yang menggunakan informasi heuristik untuk menentukan simpul mana yang akan diekspansi berikutnya. GFS termasuk pada *informed search,* karena dalam proses penjariannya memanfaatkan perkiraan jarak menuju tujuan.

Ide utama GBFS adalah menggunakan fungsi evaluasi *f(n)* untuk setiap simpul. Pada GBFS, fungsi evaluasi didefinisikan sebagai,

$$f(n) = h(n),$$

dengan:

h(n) = estimasi biaya dari simpul *n* menuju simpul tujuan.

Pada setiap langkahnya, GBFS memilih simpul yang tampak paling dekat dengan tujuan berdasarkan nilai heuristik terkecil. Oleh karena itu, algoritma ini cenderung bergerak ke arah simpul yang menurut perkiraan paling menjanjikan. Meskipun GBFS dapat mempercepat pencarian karena langsung mengarah ke simpul yang terlihat dekat dengan tujuan, algoritma ini memiliki beberapa kelemahan. GBFS dapat bersifat tidak lengkap, dapat terjebak pada *local minima* atau *plateau*, serta memiliki sifat *irrevocable*, yaitu keputusan yang telah diambil sulit diubah kembali.

Untuk implementasi pada tugas ini GBFS digunakan untuk mencari jalur dari titik awal Z menuju goal O dengan berdasarkan estimasi jarak terdekat ke goal. Berbeda dari UCS yang memilih jalur dengan total cost terkecil, GBFS memilih node yang terlihat paling menjanjikan berdasarkan nilai heuristik h(n).

```
private class Node implements Comparable<Node> {
    Tile tile;
    List<Board.Direction> path;
    int cost;
    int h;
    Board BoardCon;
}
```

tile menyimpan posisi saat ini, path menyimpan urutan gerakan yang sudah dilakukan, cost menyimpan total biaya lintasan, h menyimpan nilai heuristik, dan BoardCon menyimpan kondisi board pada state tersebut.

```
@Override
public int compareTo(Node other) {
    return Integer.compare(this.h, other.h);
}
```

Karena PriorityQueue memakai nilai h, node dengan estimasi jarak paling kecil ke goal akan dikeluarkan lebih dulu. Dengan kata lain, GBFS selalu memilih posisi yang secara heuristik terlihat paling dekat ke tujuan.

```
queue.add(new Node(startBoard.start, new ArrayList<>(),
0, calculateDistanceToGoal(startBoard.goal,
startBoard.start), startBoard));

        while (!queue.isEmpty()) {
            Node current = queue.poll();
            iterations++;
```

```
            printBoard("Iteration: " + iterations,
current.BoardCon, current.tile);

            if
(current.BoardCon.isGoalReached(current.tile)) {
                return finishRun(new Result(true,
current.path, current.cost, iterations), startTime);
            }

            Board.StateKey currentKey =
current.BoardCon.stateKey(current.tile);
            if (visited.contains(currentKey)) continue;

            visited.add(currentKey);

            for (Board.Direction dir :
Board.Direction.values()) {
                Board nextBoard =
current.BoardCon.copy();
                Tile nextStart =
nextBoard.getTile(current.tile.row, current.tile.col);
                Tile target =
nextBoard.tileIfMove(nextStart, dir);

                if (target != null &&
!visited.contains(nextBoard.stateKey(target))) {

                    int moveCost =
nextBoard.calcCost(nextStart, target, dir);
                    List<Board.Direction> nextPath = new
ArrayList<>(current.path);
                    nextPath.add(dir);

                    queue.add(new Node(target, nextPath,
current.cost + moveCost,
calculateDistanceToGoal(nextBoard.goal, target),
nextBoard));
```

Alur pathfinding GBFS:

1. Board awal disalin menggunakan board.copy().
2. Node start dimasukkan ke PriorityQueue dengan path kosong, cost 0, dan nilai heuristik dari start ke goal.

3. Program mengambil node dengan nilai h terkecil menggunakan queue.poll().
4. Jika node tersebut sudah mencapai goal dan semua tile order sudah dilewati, program mengembalikan hasil pencarian.
5. Jika belum, state ditandai sebagai visited.
6. Program mencoba empat arah gerak: UP, DOWN, LEFT, dan RIGHT.
7. Untuk setiap arah, program membuat salinan board, lalu menghitung hasil gerakan dengan tileIfMove.
8. Jika gerakan valid, program menghitung cost gerakan, menambahkan arah ke path, menghitung heuristik baru, lalu memasukkan node baru ke queue.

Pada program ini, satu aksi bergerak terus ke satu arah sampai berhenti atau *sliding* sampai tidak dapat bergerak atau pergerakan tidak valid. Aturan ini diatur oleh tileIfMove. Walaupun cost tetap dihitung oleh calcCost GBFS tidak menggunakan cost sebagai prioritas utama. Cost hanya disimpan untuk hasil akhir, sedangkan pemilihan jalur tetap berdasarkan nilai heuristik h. Karena itu, GBFS biasanya lebih cepat mengarah ke goal, tetapi tidak menjamin jalur dengan total cost minimum.

5. A*

A* adalah algoritma pencarian yang menggabungkan biaya aktual dan estimasi biaya menuju tujuan untuk pencariannya, Ide utama algoritma ini adalah menghindari ekspansi jalur yang sudah terlihat mahal. Dengan kata lain, A* memilih simpul berdasarkan perkiraan total biaya jalur dari simpul awal, melewati simpul tersebut, sampai ke simpul tujuan.
Fungsi evaluasi pada A* adalah:

$$f(n) = g(n) + h(n)$$

Dengan:
$g(n)$ = biaya dari simpul awal menuju simpul n
$h(n)$ = estimasi biaya dari simpul n menuju simpul tujuan
$f(n)$ = estimasi total biaya jalur dari awal ke tujuan melalui simpul n

Nilai $g(n)$ menunjukkan biaya aktual yang telah dikeluarkan dari simpul awal sampai simpul n, sedangkan $h(n)$ menunjukkan perkiraan biaya yang diperlukan untuk mencapai simpul tujuan. Dengan menjumlahkan kedua nilai tersebut, A* dapat mempertimbangkan biaya yang sudah pasti dan estimasi biaya ke depan secara bersamaan.

Untuk implementasi pada tugas ini A* digunakan untuk mencari jalur dari titik awal Z menuju goal O dengan menggabungkan biaya aktual dan estimasi biaya untuk menuju *n.*

```
private class Node implements Comparable<Node>{
    Tile tile;
    List<Board.Direction> path;
    int cost;
    int h;
```

```
    Board BoardCon;
}
```

tile menyimpan posisi agen saat ini, path menyimpan urutan gerakan yang sudah dilakukan, cost adalah total biaya dari start sampai posisi tersebut, h adalah nilai heuristik menuju goal, dan BoardCon menyimpan kondisi board pada state tersebut.

```
@Override
public int compareTo(Node other) {
    int thisf = this.cost + this.h;
    int otherf = other.cost + other.h;
    return Integer.compare(thisf, otherf);
}
```

Karena node dimasukkan ke PriorityQueue, node dengan nilai f(n) terkecil akan diproses lebih dulu. Artinya, A* tidak hanya memilih jalur yang murah sejauh ini, tetapi juga mempertimbangkan seberapa dekat state tersebut dengan goal.

```
           queue.add(new Node(startBoard.start, new
ArrayList<>(),0, calculateDistanceToGoal(startBoard.goal,
startBoard.start), startBoard));

            while (!queue.isEmpty()) {
                Node current = queue.poll();
                iterations++;

                printBoard("Iteration: " + iterations,
current.BoardCon, current.tile);

                if
(current.BoardCon.isGoalReached(current.tile)) {
                    return finishRun(new Result(true,
current.path, current.cost, iterations), startTime);
                }

                Board.StateKey currentKey =
current.BoardCon.stateKey(current.tile);
                if (visited.contains(currentKey))
continue;

                visited.add(currentKey);
```

```
                for (Board.Direction dir :
Board.Direction.values()) {
                    Board nextBoard =
current.BoardCon.copy();
                    Tile nextStart =
nextBoard.getTile(current.tile.row, current.tile.col);
                    Tile target =
nextBoard.tileIfMove(nextStart, dir);

                    if (target != null &&
!visited.contains(nextBoard.stateKey(target))) {

                        int moveCost =
nextBoard.calcCost(nextStart, target, dir);
                        List<Board.Direction> nextPath =
new ArrayList<>(current.path);
                        nextPath.add(dir);
                        queue.add(new Node(target,
nextPath, current.cost + moveCost,
calculateDistanceToGoal(nextBoard.goal, target),
nextBoard));
```

Alur pathfinding A*:

1. Program menyalin board awal menggunakan board.copy().
2. Node start dimasukkan ke PriorityQueue dengan cost = 0 dan h dari start ke goal.
3. Program mengambil node dengan nilai cost + h terkecil.
4. Jika node tersebut sudah mencapai goal dan semua tile order sudah dilewati, pencarian selesai.
5. Jika belum, state saat ini dicek pada visited.
6. Program mencoba empat arah gerak: UP, DOWN, LEFT, dan RIGHT.
7. Untuk setiap arah, program membuat salinan board baru.
8. Program menentukan tile tujuan gerakan dengan tileIfMove.
9. Jika gerakan valid, program menghitung biaya gerakan dengan calcCost.
10. Program membuat path baru, menghitung cost + moveCost, menghitung heuristik baru, lalu memasukkan node tersebut ke priority queue.

    Pada program ini, satu aksi bergerak terus ke satu arah sampai berhenti atau *sliding* sampai tidak dapat bergerak atau pergerakan tidak valid. Aturan ini diatur oleh tileIfMove dan biaya gerakan dihitung oleh calcCost, nilai calcCost ini yang menjadi *g(n)* pada A*.

6. Implementasi Heuristik

a. Manhattan Distance

Manhattan Distance menghitung estimasi jarak berdasarkan jumlah selisih baris dan kolom antara posisi saat ini dan goal.

```
private int Manhattan(Tile goal, Tile curr) {
    return Math.abs(goal.row - curr.row) +
Math.abs(goal.col - curr.col);
}
```

b. Euclidean Distance

Euclidean Distance menghitung jarak garis lurus antara posisi saat ini dan goal.

```
private int Euclidean(Tile goal, Tile curr) {
    int dr = goal.row - curr.row;
    int dc = goal.col - curr.col;
    return (int) Math.sqrt(dr * dr + dc * dc);
}
```

c. Chebysev Distance

Chebyshev Distance mengambil nilai terbesar antara selisih baris dan selisih kolom.

```
private int Chebyshev(Tile goal, Tile curr) {
    return Math.max(Math.abs(goal.row - curr.row),
Math.abs(goal.col - curr.col));
}
```

Suatu heuristik disebut admissible jika nilai estimasinya tidak lebih besar daripada cost aktual minimum dari suatu state menuju goal. Dengan kata lain, heuristik tidak boleh melakukan overestimate terhadap biaya sebenarnya.

Jika setiap tile yang dapat dilewati memiliki cost minimal 1, maka Manhattan, Euclidean, dan Chebyshev dapat dianggap admissible terhadap cost lintasan. Manhattan menghitung estimasi berdasarkan selisih baris dan kolom. Euclidean menghitung jarak garis lurus, sedangkan Chebyshev mengambil selisih terbesar antara selisih baris dan kolom. Nilai Euclidean dan Chebyshev selalu lebih kecil atau sama dengan Manhattan, sehingga jika Manhattan tidak melebihi cost aktual, maka Euclidean dan Chebyshev juga tidak melebihi cost aktual.

Namun, jika terdapat tile dengan cost 0, maka sifat admissible tidak dapat dijamin. Hal ini karena cost aktual dari suatu state menuju goal bisa saja bernilai lebih kecil daripada estimasi heuristik. Misalnya, jika posisi saat ini masih berjarak beberapa petak dari goal, heuristik Manhattan, Euclidean, maupun Chebyshev akan menghasilkan nilai lebih besar dari 0. Akan tetapi, jika semua tile yang dilewati menuju goal memiliki cost 0, maka cost aktual lintasannya

adalah 0. Dalam kasus tersebut, nilai heuristik lebih besar daripada cost sebenarnya, sehingga heuristik melakukan overestimate dan tidak admissible.

Karena itu, admissibility ketiga heuristik sangat bergantung pada batas bawah cost tile. Jika semua tile yang dapat dilewati memiliki cost minimal 1, maka ketiga heuristik dapat dianggap admissible.

7. Perbandingan Algotima

UCS memilih node dengan total cost terkecil dari start. Jika semua cost bernilai non-negatif, UCS dapat menjamin solusi optimal, yaitu rute dengan total cost paling kecil.

Kelemahannya adalah UCS tidak memiliki informasi arah menuju goal. Algoritma ini hanya tahu cost yang sudah dikeluarkan, sehingga dapat mengeksplorasi banyak kemungkinan jalur yang murah tetapi tidak mengarah ke tujuan. Pada puzzle berukuran besar, jumlah state yang dikunjungi bisa menjadi sangat banyak.

GBFS memilih node berdasarkan nilai heuristik terkecil, yaitu node yang terlihat paling dekat dengan goal. Pada implementasi ini, heuristik menghitung jarak posisi saat ini ke goal. Karena GBFS hanya melihat *h(n)*, algoritma ini biasanya lebih cepat mengarah ke tujuan dibanding UCS.

Namun, GBFS tidak menjamin solusi optimal. Jalur yang secara koordinat terlihat dekat ke goal belum tentu memiliki cost rendah. Dalam Ice Sliding Puzzle, hal ini penting karena pemain tidak bisa berhenti di sembarang tempat, melainkan terus meluncur sampai terhalang. Akibatnya, estimasi jarak ke goal bisa menyesatkan jika ada dinding, lava, checkpoint berurutan, atau tile dengan cost tinggi.

A* lebih efisien daripada UCS jika heuristik yang digunakan cukup informatif. A* juga lebih aman daripada GBFS karena tetap memperhitungkan cost aktual. Jika heuristik bersifat admissible dan konsisten, serta cost tidak negatif, A* dapat menghasilkan solusi optimal.

BAB II

IMPLEMENTASI PROGRAM

1. Tile.java

```
package backend;

public class Tile {
    enum Type {
        P,
        X,
        L,
        O,
        Z
    }

    public int row, col;
    public Type type;
    public int order = -1;
    public int cost;
    public boolean hasBeenPassed;
    public Tile right, left, up, down;
    public Tile previousOrder, nextOrder;

    public Tile(int row, int col, Type type) {
        this.row = row;
        this.col = col;
        this.type = type;
        this.hasBeenPassed = false;
    }

    public Boolean canEnter() {
        if (order != -1 && !hasBeenPassed) {
            return hasPassedPrevious();
        }
        return true;
    }

    public void setOrder(int order) {
        this.order = order;
    }

    public void setCost(int cost) {
```

```
        this.cost = cost;
    }

    public void setRight(Tile right) {
        this.right = right;
    }

    public void setLeft(Tile left) {
        this.left = left;
    }

    public void setUp(Tile up) {
        this.up = up;
    }

    public void setDown(Tile down) {
        this.down = down;
    }

    public void setPreviousOrder(Tile previousOrder) {
        this.previousOrder = previousOrder;
    }

    public void setNextOrder(Tile nextOrder) {
        this.nextOrder = nextOrder;
    }

    public boolean hasPassedPrevious() {
        if (order == 0 || order == -1) {
            return true;
        }
        return previousOrder != null &&
previousOrder.hasBeenPassed;
    }


    public Tile nextRight() { return right; }
    public Tile nextLeft() { return left; }
    public Tile nextUp() { return up; }
    public Tile nextDown() { return down; }

    public boolean isWall() { return type == Type.X; }
    public boolean isLava() { return type == Type.L; }
    public boolean isGoal() { return type == Type.O; }
```

```
    public boolean isStart() { return type == Type.Z; }
    public boolean isPath() { return type == Type.P; }


}
```

2. Board.java

```
package backend;

public class Board {
    public record StateKey(int row, int col, int
lastPassedOrder) {
    }

    public int row, col;
    public Tile[][] tiles;
    public Tile[] orderTiles;
    public Tile start;
    public Tile goal;
    public int maxOrder;
    public Tile firstTargetOrder;
    public Tile currentTile;
    public boolean considerOrder = true;

    public enum Direction {
        UP,
        DOWN,
        LEFT,
        RIGHT
    }

    public Board(int row, int col) {
        this.row = row;
        this.col = col;
        this.tiles = new Tile[row][col];
    }

    public Tile getTile(int row, int col) {
        return tiles[row][col];
    }
```

```
    public Board copy() {
        Board copy = new Board(row, col);

        copy.maxOrder = maxOrder;
        copy.considerOrder = considerOrder;

        for (int r = 0; r < row; r++) {
            for (int c = 0; c < col; c++) {
                Tile oldTile = tiles[r][c];
                Tile newTile = new Tile(r, c, oldTile.type);

                newTile.order = oldTile.order;
                newTile.cost = oldTile.cost;
                newTile.hasBeenPassed = oldTile.hasBeenPassed;

                copy.tiles[r][c] = newTile;

                if (oldTile == start) {
                    copy.start = newTile;
                }
                if (oldTile == goal) {
                    copy.goal = newTile;
                }
                if (oldTile == currentTile) {
                    copy.currentTile = newTile;
                }
            }
        }

        for (int r = 0; r < row; r++) {
            for (int c = 0; c < col; c++) {
                Tile tile = copy.tiles[r][c];
                tile.setUp(r > 0 ? copy.tiles[r - 1][c] :
null);
                tile.setDown(r + 1 < row ? copy.tiles[r +
1][c] : null);
                tile.setLeft(c > 0 ? copy.tiles[r][c - 1] :
null);
                tile.setRight(c + 1 < col ? copy.tiles[r][c +
1] : null);
            }
        }

        Tile[] orderedTiles = new Tile[maxOrder + 1];
```

```
        for (int r = 0; r < row; r++) {
            for (int c = 0; c < col; c++) {
                Tile tile = copy.tiles[r][c];
                if (tile.order >= 0 && tile.order <
orderedTiles.length) {
                    orderedTiles[tile.order] = tile;
                }
            }
        }

        copy.orderTiles = orderedTiles;
        copy.firstTargetOrder = orderedTiles.length > 0 ?
orderedTiles[0] : null;

        for (int i = 0; i < orderedTiles.length; i++) {
            if (orderedTiles[i] == null) {
                continue;
            }

            orderedTiles[i].setPreviousOrder(i > 0 ?
orderedTiles[i - 1] : null);
            orderedTiles[i].setNextOrder(i + 1 <
orderedTiles.length ? orderedTiles[i + 1] : null);
        }

        return copy;
    }

    public String printBoard() {
        StringBuilder builder = new StringBuilder();
        Tile playerTile = currentTile != null ? currentTile :
start;

        for (int r = 0; r < row; r++) {
            for (int c = 0; c < col; c++) {
                Tile tile = tiles[r][c];

                if (tile == playerTile) {
                    builder.append('Z');
                } else if (tile.order >= 0) {
                    builder.append(tile.order);
                } else if (tile.isWall()) {
                    builder.append('X');
                } else if (tile.isLava()) {
```

```
                    builder.append('L');
                } else if (tile.isGoal()) {
                    builder.append('O');
                } else {
                    builder.append('*');
                }
            }
            builder.append(System.lineSeparator());
        }

        return builder.toString();
    }

    public Tile tileIfMove(Tile startTile, Direction dir) {
        Tile temp = startTile;

        while (temp != null) {
            Tile nextTile = nextTile(temp, dir);

            if (nextTile == null) {
                return null;
            }
            if (nextTile.isWall()) {
                break;
            }

            if (nextTile.isLava()) {
                return null;
            }
            if (!nextTile.canEnter()) {
                return null;
            }
            if (nextTile.order != -1) {
                nextTile.hasBeenPassed = true;
            }
            temp = nextTile;
        }
        return temp;
    }

    public int calcCost(Tile startTile, Tile endTile,
Direction dir) {
        int cost = 0;
        Tile temp = startTile;
```

```
        while (temp != endTile) {
            temp = nextTile(temp, dir);
            cost += temp.cost;
        }
        return cost;
    }

    private Tile nextTile(Tile tile, Direction dir) {
        return switch (dir) {
            case UP -> tile.up;
            case DOWN -> tile.down;
            case LEFT -> tile.left;
            case RIGHT -> tile.right;
        };
    }

    public void printOrder() {
        while (firstTargetOrder != null) {
            System.out.println("Order " +
firstTargetOrder.order + ": (" + firstTargetOrder.row + ", "
                    + firstTargetOrder.col + ")");
            firstTargetOrder = firstTargetOrder.nextOrder;
        }
    }

    public boolean isGoalReached(Tile currentTile) {
        if (currentTile != goal) {
            return false;
        }

        if (!considerOrder) {
            return true;
        }

        return hasPassedOrders();
    }

    public boolean hasPassedOrders() {
        for (int r = 0; r < row; r++) {
            for (int c = 0; c < col; c++) {
                Tile tile = tiles[r][c];
                if (tile.order != -1 && !tile.hasBeenPassed) {
                    return false;
```

```
                }
            }
        }

        return true;
    }

    public StateKey stateKey(Tile currentTile) {
        if (!considerOrder) {
            return new StateKey(currentTile.row,
currentTile.col, -1);
        }

        return new StateKey(currentTile.row, currentTile.col,
lastPassedOrder());
    }

    private int lastPassedOrder() {
        int lastPassedOrder = -1;

        if (orderTiles != null) {
            for (Tile tile : orderTiles) {
                if (tile == null || !tile.hasBeenPassed) {
                    break;
                }
                lastPassedOrder = tile.order;
            }
            return lastPassedOrder;
        }

        for (int r = 0; r < row; r++) {
            for (int c = 0; c < col; c++) {
                Tile tile = tiles[r][c];
                if (tile.order > lastPassedOrder &&
tile.hasBeenPassed) {
                    lastPassedOrder = tile.order;
                }
            }
        }

        return lastPassedOrder;
    }
}
```

3. **Algorithm.java**

```
package backend;

public abstract class Algorithm {
    private StringBuilder logger = new StringBuilder();
    private StringBuilder output = new StringBuilder();
    private long time;

    public abstract Result solve(Board board);

    protected long clearLog() {
        logger.setLength(0);
        time = 0;
        return System.nanoTime();
    }

    protected void clearOutput() {
        output.setLength(0);
    }

    protected Result finishRun(Result result, long start) {
        time = (System.nanoTime() - start) / 1_000_000;
        return result;
    }

    protected void log(String message) {
        logger.append(message).append(System.lineSeparator());
    }

    protected void printBoard(String title, Board board, Tile
currentTile) {
        if (board == null) {
            return;
        }

        board.currentTile = currentTile;
        output.append(title).append(System.lineSeparator());

output.append(board.printBoard()).append(System.lineSeparator(
));
    }
```

```
    public String getLog() {
        return logger.toString();
    }

    public String getOutput() {
        return output.toString();
    }

    public long getTime() {
        return time;
    }
}
```

4. **UCS.java**

```
public class UCS extends Algorithm {
    private class Node implements Comparable<Node> {
        Tile tile;
        List<Board.Direction> path;
        int cost;
        Board BoardCon;

        Node(Tile tile, List<Board.Direction> path, int cost,
Board BoardCon) {
            this.tile = tile;
            this.path = path;
            this.cost = cost;
            this.BoardCon = BoardCon;
        }

        @Override
        public int compareTo(Node other) {
            return Integer.compare(this.cost, other.cost);
        }
    }

    @Override
    public Result solve(Board board) {
        long startTime = clearLog();
        clearOutput();
        PriorityQueue<Node> queue = new PriorityQueue<>();
        Set<Board.StateKey> visited = new HashSet<>();
        int iterations = 0;
        Board startBoard = board.copy();
```

```
        queue.add(new Node(startBoard.start, new
ArrayList<>(), 0, startBoard));

        while (!queue.isEmpty()) {
            Node current = queue.poll();
            iterations++;

            printBoard("Iteration: " + iterations,
current.BoardCon, current.tile);

            if (current.BoardCon.isGoalReached(current.tile))
{
                return finishRun(new Result(true,
current.path, current.cost, iterations), startTime);
            }

            Board.StateKey currentKey =
current.BoardCon.stateKey(current.tile);
            if (visited.contains(currentKey)) {
                continue;
            }
            visited.add(currentKey);

            for (Board.Direction dir :
Board.Direction.values()) {
                Board nextBoard = current.BoardCon.copy();
                Tile nextStart =
nextBoard.getTile(current.tile.row, current.tile.col);
                Tile target = nextBoard.tileIfMove(nextStart,
dir);

                if (target != null &&
!visited.contains(nextBoard.stateKey(target))) {

                    int moveCost =
nextBoard.calcCost(nextStart, target, dir);
                    List<Board.Direction> nextPath = new
ArrayList<>(current.path);
                    nextPath.add(dir);

                    queue.add(new Node(target, nextPath,
current.cost + moveCost, nextBoard));
                    log("Itteration: " + iterations + ",
```

```
Current Tile: (" + target.row + ", " + target.col + "), Cost:
" + (current.cost + moveCost) + ", Path" + nextPath);
                }
            }
        }

        return finishRun(new Result(false,
Collections.emptyList(), 0, iterations), startTime);
    }
}
```

5. **GBFS.java**

```
public class GBFS extends Algorithm {
    private Heuristic heuristic = Heuristic.Manhattan;

    public void setHeuristic(Heuristic h) {
        this.heuristic = h;
    }

    private class Node implements Comparable<Node> {
        Tile tile;
        List<Board.Direction> path;
        int cost;
        int h;
        Board BoardCon;

        Node(Tile tile, List<Board.Direction> path, int cost,
int h, Board BoardCon) {
            this.tile = tile;
            this.path = path;
            this.cost = cost;
            this.h = h;
            this.BoardCon = BoardCon;
        }

        @Override
        public int compareTo(Node other) {
            return Integer.compare(this.h, other.h);
        }
    }
```

```
    public Result solve(Board board) {
        long startTime = clearLog();
        clearOutput();
        PriorityQueue<Node> queue = new PriorityQueue<>();
        Set<Board.StateKey> visited = new HashSet<>();
        int iterations = 0;
        Board startBoard = board.copy();

        queue.add(new Node(startBoard.start, new
ArrayList<>(), 0, calculateDistanceToGoal(startBoard.goal,
startBoard.start), startBoard));

        while (!queue.isEmpty()) {
            Node current = queue.poll();
            iterations++;

            printBoard("Iteration: " + iterations,
current.BoardCon, current.tile);

            if (current.BoardCon.isGoalReached(current.tile))
{
                return finishRun(new Result(true,
current.path, current.cost, iterations), startTime);
            }

            Board.StateKey currentKey =
current.BoardCon.stateKey(current.tile);
            if (visited.contains(currentKey)) continue;

            visited.add(currentKey);

            for (Board.Direction dir :
Board.Direction.values()) {
                Board nextBoard = current.BoardCon.copy();
                Tile nextStart =
nextBoard.getTile(current.tile.row, current.tile.col);
                Tile target = nextBoard.tileIfMove(nextStart,
dir);

                if (target != null &&
!visited.contains(nextBoard.stateKey(target))) {

                    int moveCost =
nextBoard.calcCost(nextStart, target, dir);
```

```
                    List<Board.Direction> nextPath = new
ArrayList<>(current.path);
                    nextPath.add(dir);

                    queue.add(new Node(target, nextPath,
current.cost + moveCost,
calculateDistanceToGoal(nextBoard.goal, target), nextBoard));
                    log("Itteration: " + iterations + ",
Current Tile: (" + target.row + ", " + target.col + "), Cost:
" + (current.cost + moveCost) + ", Path" + nextPath);
                }
            }
        }

        return finishRun(new Result(false,
Collections.emptyList(), 0, iterations), startTime);
    }

    private int calculateDistanceToGoal(Tile goal, Tile curr)
{
        return switch (heuristic) {
            case Manhattan -> Manhattan(goal, curr);
            case Euclidean -> Euclidean(goal, curr);
            case Chebyshev -> Chebyshev(goal, curr);
        };
    }

    private int Manhattan(Tile goal, Tile curr) {
        return Math.abs(goal.row - curr.row) +
Math.abs(goal.col - curr.col);
    }

    private int Euclidean(Tile goal, Tile curr) {
        int dr = goal.row - curr.row;
        int dc = goal.col - curr.col;
        return (int) Math.sqrt(dr * dr + dc * dc);
    }

    private int Chebyshev(Tile goal, Tile curr) {
        return Math.max(Math.abs(goal.row - curr.row),
Math.abs(goal.col - curr.col));
    }
}
```

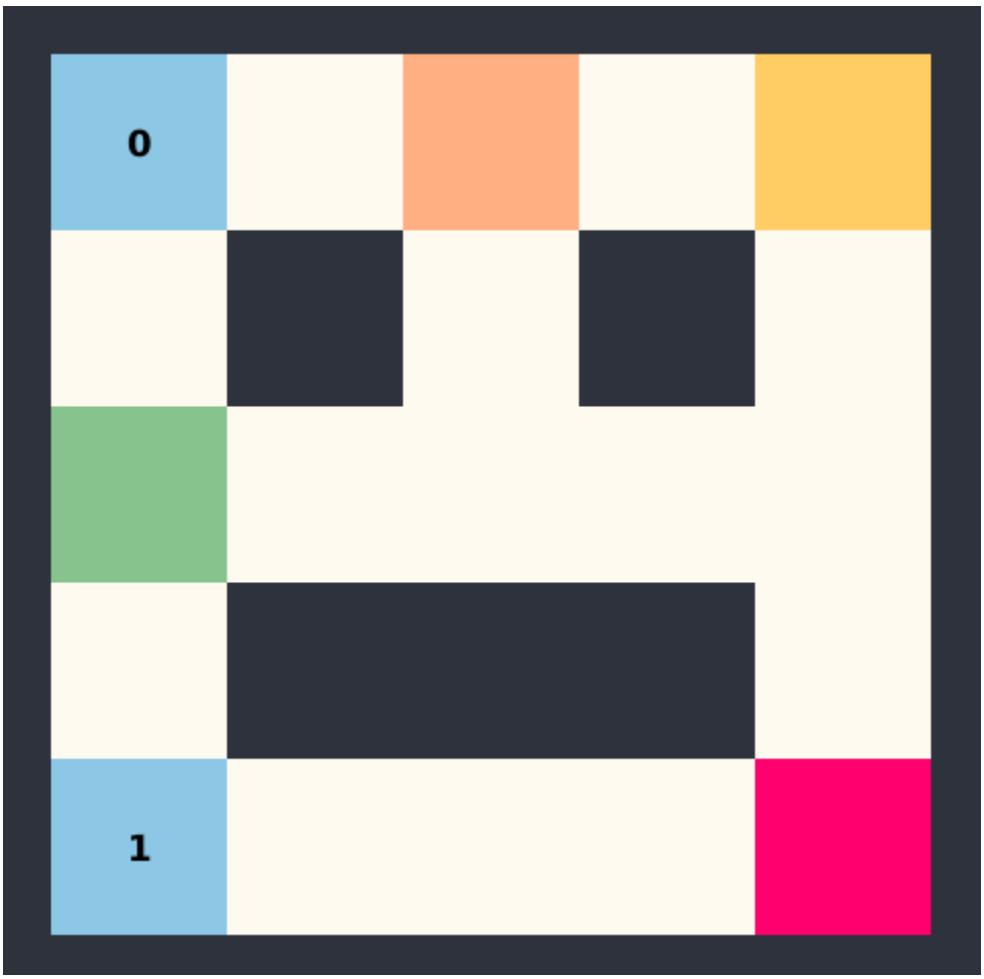
6. Astar.java

```
public class AStar extends Algorithm {
    public enum Heuristic {
        Manhattan,  // Manhattan
        Euclidean,  // Euclidean
        Chebyshev   // Chebyshev
    }

    private Heuristic heuristic = Heuristic.Manhattan;

    private class Node implements Comparable<Node>{
        Tile tile;
        List<Board.Direction> path;
        int cost;
        int h;
        Board BoardCon;

        Node(Tile tile, List<Board.Direction> path, int cost,
int h, Board BoardCon) {
            this.tile = tile;
            this.path = path;
            this.cost = cost;
            this.h = h;
            this.BoardCon = BoardCon;
        }

        @Override
        public int compareTo(Node other) {
            int thisf = this.cost + this.h;
            int otherf = other.cost + other.h;
            return Integer.compare(thisf, otherf);
        }

    }

    public void setHeuristic(Heuristic h) {
        this.heuristic = h;
    }

    @Override
    public Result solve(Board board) {
            long startTime = clearLog();
            clearOutput();
```

```
            PriorityQueue<Node> queue = new PriorityQueue<>();
            Set<Board.StateKey> visited = new HashSet<>();
            int iterations = 0;
            Board startBoard = board.copy();

            queue.add(new Node(startBoard.start, new
ArrayList<>(),0, calculateDistanceToGoal(startBoard.goal,
startBoard.start), startBoard));

            while (!queue.isEmpty()) {
                Node current = queue.poll();
                iterations++;

                printBoard("Iteration: " + iterations,
current.BoardCon, current.tile);

                if
(current.BoardCon.isGoalReached(current.tile)) {
                    return finishRun(new Result(true,
current.path, current.cost, iterations), startTime);
                }

                Board.StateKey currentKey =
current.BoardCon.stateKey(current.tile);
                if (visited.contains(currentKey)) continue;

                visited.add(currentKey);

                for (Board.Direction dir :
Board.Direction.values()) {
                    Board nextBoard = current.BoardCon.copy();
                    Tile nextStart =
nextBoard.getTile(current.tile.row, current.tile.col);
                    Tile target =
nextBoard.tileIfMove(nextStart, dir);

                    if (target != null &&
!visited.contains(nextBoard.stateKey(target))) {

                        int moveCost =
nextBoard.calcCost(nextStart, target, dir);
                        List<Board.Direction> nextPath = new
ArrayList<>(current.path);
                        nextPath.add(dir);
```

```
                        queue.add(new Node(target, nextPath,
current.cost + moveCost,
calculateDistanceToGoal(nextBoard.goal, target), nextBoard));
                        log("Itteration: " + iterations + ",
Current Tile: (" + target.row + ", " + target.col + "), Cost:
" + (current.cost + moveCost) + ", Path" + nextPath);
                    }
                }
            }

        return finishRun(new Result(false,
Collections.emptyList(), 0, iterations), startTime);
    }

    private int calculateDistanceToGoal(Tile goal, Tile curr)
{
        return switch (heuristic) {
            case Manhattan -> Manhattan(goal, curr);
            case Euclidean -> Euclidean(goal, curr);
            case Chebyshev -> Chebyshev(goal, curr);
        };
    }

    private int Manhattan(Tile goal, Tile curr) {
        return Math.abs(goal.row - curr.row) +
Math.abs(goal.col - curr.col);
    }

    private int Euclidean(Tile goal, Tile curr) {
        int dr = goal.row - curr.row;
        int dc = goal.col - curr.col;
        return (int) Math.sqrt(dr * dr + dc * dc);
    }

    private int Chebyshev(Tile goal, Tile curr) {
        return Math.max(Math.abs(goal.row - curr.row),
Math.abs(goal.col - curr.col));
    }
}
```

BAB III
PENGUJIAN



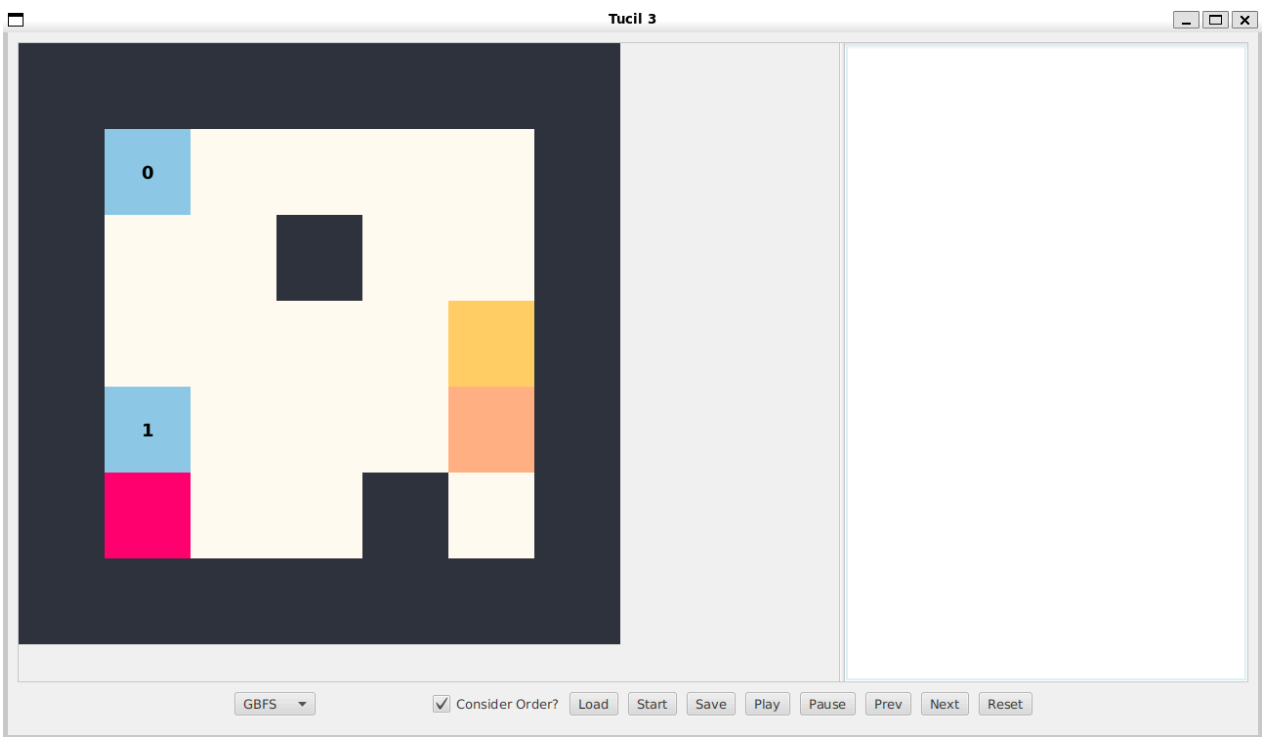
Keterangan

0 Ordered Tile	Start Position	Posisi Player
Lava	Goal	Wall/ Tembok

Test Case 1

7 7
XXXXXXX
X0**X**
XX**X**
X**OX**
X1*LX**
XZX*X**
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999

Hasil



UCS

Found: true
Moves: [RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, RIGHT]
Total Cost: 87
Iterations: 15
Time: 77 ms

Iteration Log
Itteration: 1, Current Tile: (5, 3), Cost: 7, Path[RIGHT]
Itteration: 2, Current Tile: (3, 3), Cost: 17, Path[RIGHT, UP]
Itteration: 3, Current Tile: (3, 1), Cost: 27, Path[RIGHT, UP, LEFT]
Itteration: 3, Current Tile: (3, 5), Cost: 26, Path[RIGHT, UP, RIGHT]
Itteration: 4, Current Tile: (1, 5), Cost: 36, Path[RIGHT, UP, RIGHT, UP]
Itteration: 4, Current Tile: (3, 1), Cost: 44, Path[RIGHT, UP, RIGHT, LEFT]
Itteration: 5, Current Tile: (1, 1), Cost: 37, Path[RIGHT, UP, LEFT, UP]
Itteration: 6, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, RIGHT, UP, LEFT]
Itteration: 7, Current Tile: (5, 1), Cost: 61, Path[RIGHT, UP, LEFT, UP, DOWN]
Itteration: 7, Current Tile: (1, 5), Cost: 53, Path[RIGHT, UP, LEFT, UP, RIGHT]
Itteration: 11, Current Tile: (1, 1), Cost: 79, Path[RIGHT, UP, LEFT, UP, DOWN, UP]
Itteration: 11, Current Tile: (5, 3), Cost: 68, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT]
Itteration: 12, Current Tile: (3, 3), Cost: 78, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP]
Itteration: 13, Current Tile: (3, 1), Cost: 88, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, LEFT]
Itteration: 13, Current Tile: (3, 5), Cost: 87, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, RIGHT]
Itteration: 14, Current Tile: (1, 5), Cost: 95, Path[RIGHT, UP, LEFT, UP, DOWN, UP, RIGHT]

Iteration
Iteration: 1
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 2
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 3
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 4
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Iteration: 5
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 6
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 7
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX

X***X*X
XXXXXXX

Iteration: 8
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 9
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 10
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 11
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 12
XXXXXXX
X0****X
X**X**X

X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 13
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 14
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 15
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

GBFS - Manhattan

Found: true
Moves: [RIGHT, UP, RIGHT, LEFT, UP, DOWN, RIGHT, UP, RIGHT]
Total Cost: 104
Iterations: 14
Time: 15 ms

Iteration Log
Itteration: 1, Current Tile: (5, 3), Cost: 7, Path[RIGHT]
Itteration: 2, Current Tile: (3, 3), Cost: 17, Path[RIGHT, UP]
Itteration: 3, Current Tile: (3, 1), Cost: 27, Path[RIGHT, UP, LEFT]

Itteration: 3, Current Tile: (3, 5), Cost: 26, Path[RIGHT, UP, RIGHT]
Itteration: 4, Current Tile: (1, 5), Cost: 36, Path[RIGHT, UP, RIGHT, UP]
Itteration: 4, Current Tile: (3, 1), Cost: 44, Path[RIGHT, UP, RIGHT, LEFT]
Itteration: 5, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, RIGHT, UP, LEFT]
Itteration: 6, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, RIGHT, LEFT, UP]
Itteration: 8, Current Tile: (5, 1), Cost: 78, Path[RIGHT, UP, RIGHT, LEFT, UP, DOWN]
Itteration: 8, Current Tile: (1, 5), Cost: 70, Path[RIGHT, UP, RIGHT, LEFT, UP, RIGHT]
Itteration: 11, Current Tile: (1, 1), Cost: 96, Path[RIGHT, UP, RIGHT, LEFT, UP, DOWN, UP]
Itteration: 11, Current Tile: (5, 3), Cost: 85, Path[RIGHT, UP, RIGHT, LEFT, UP, DOWN, RIGHT]
Itteration: 12, Current Tile: (3, 3), Cost: 95, Path[RIGHT, UP, RIGHT, LEFT, UP, DOWN, RIGHT, UP]
Itteration: 13, Current Tile: (3, 1), Cost: 105, Path[RIGHT, UP, RIGHT, LEFT, UP, DOWN, RIGHT, UP, LEFT]
Itteration: 13, Current Tile: (3, 5), Cost: 104, Path[RIGHT, UP, RIGHT, LEFT, UP, DOWN, RIGHT, UP, RIGHT]

Iteration
Iteration: 1
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 2
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 3
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 4
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Iteration: 5
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 6
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 7
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 8
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX

```
X***X*X
XXXXXXX

Iteration: 9
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 10
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 11
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 12
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 13
XXXXXXX
X0****X
X**X**X
```

X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 14
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

GBFS - Euclidian

Found: true
Moves: [RIGHT, UP, RIGHT, UP, LEFT, DOWN, RIGHT, UP, RIGHT]
Total Cost: 104
Iterations: 13
Time: 2 ms

Iteration Log
Itteration: 1, Current Tile: (5, 3), Cost: 7, Path[RIGHT]
Itteration: 2, Current Tile: (3, 3), Cost: 17, Path[RIGHT, UP]
Itteration: 3, Current Tile: (3, 1), Cost: 27, Path[RIGHT, UP, LEFT]
Itteration: 3, Current Tile: (3, 5), Cost: 26, Path[RIGHT, UP, RIGHT]
Itteration: 4, Current Tile: (1, 5), Cost: 36, Path[RIGHT, UP, RIGHT, UP]
Itteration: 4, Current Tile: (3, 1), Cost: 44, Path[RIGHT, UP, RIGHT, LEFT]
Itteration: 5, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, RIGHT, UP, LEFT]
Itteration: 6, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, RIGHT, LEFT, UP]
Itteration: 7, Current Tile: (5, 1), Cost: 78, Path[RIGHT, UP, RIGHT, UP, LEFT, DOWN]
Itteration: 7, Current Tile: (1, 5), Cost: 70, Path[RIGHT, UP, RIGHT, UP, LEFT, RIGHT]
Itteration: 10, Current Tile: (1, 1), Cost: 96, Path[RIGHT, UP, RIGHT, UP, LEFT, DOWN, UP]
Itteration: 10, Current Tile: (5, 3), Cost: 85, Path[RIGHT, UP, RIGHT, UP, LEFT, DOWN, RIGHT]
Itteration: 11, Current Tile: (3, 3), Cost: 95, Path[RIGHT, UP, RIGHT, UP, LEFT, DOWN, RIGHT, UP]
Itteration: 12, Current Tile: (3, 1), Cost: 105, Path[RIGHT, UP, RIGHT, UP, LEFT, DOWN, RIGHT, UP, LEFT]
Itteration: 12, Current Tile: (3, 5), Cost: 104, Path[RIGHT, UP, RIGHT, UP, LEFT, DOWN, RIGHT, UP, RIGHT]

Iteration
Iteration: 1

```
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 2
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 3
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 4
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Iteration: 5
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX
```

Iteration: 6
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 7
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 8
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 9
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 10
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX

XZ**X*X
XXXXXXX

Iteration: 11
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 12
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 13
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

A* - Manhattan

Found: true
Moves: [RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, RIGHT]
Total Cost: 87
Iterations: 15
Time: 10 ms

Iteration Log
Itteration: 1, Current Tile: (5, 3), Cost: 7, Path[RIGHT]
Itteration: 2, Current Tile: (3, 3), Cost: 17, Path[RIGHT, UP]
Itteration: 3, Current Tile: (3, 1), Cost: 27, Path[RIGHT, UP, LEFT]
Itteration: 3, Current Tile: (3, 5), Cost: 26, Path[RIGHT, UP, RIGHT]
Itteration: 4, Current Tile: (1, 5), Cost: 36, Path[RIGHT, UP, RIGHT, UP]

```
Itteration: 4, Current Tile: (3, 1), Cost: 44, Path[RIGHT, UP, RIGHT, LEFT]
Itteration: 5, Current Tile: (1, 1), Cost: 37, Path[RIGHT, UP, LEFT, UP]
Itteration: 6, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, RIGHT, UP, LEFT]
Itteration: 7, Current Tile: (5, 1), Cost: 61, Path[RIGHT, UP, LEFT, UP, DOWN]
Itteration: 7, Current Tile: (1, 5), Cost: 53, Path[RIGHT, UP, LEFT, UP, RIGHT]
Itteration: 11, Current Tile: (1, 1), Cost: 79, Path[RIGHT, UP, LEFT, UP, DOWN, UP]
Itteration: 11, Current Tile: (5, 3), Cost: 68, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT]
Itteration: 12, Current Tile: (3, 3), Cost: 78, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP]
Itteration: 13, Current Tile: (3, 1), Cost: 88, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, LEFT]
Itteration: 13, Current Tile: (3, 5), Cost: 87, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, RIGHT]
Itteration: 14, Current Tile: (1, 5), Cost: 95, Path[RIGHT, UP, LEFT, UP, DOWN, UP, RIGHT]


Iteration
Iteration: 1
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 2
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 3
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 4
XXXXXXX
```

X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Iteration: 5
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 6
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 7
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 8
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 9
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 10
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 11
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 12
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 13
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X

XXXXXXX

Iteration: 14
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 15
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

A* - Chebyshev

Found: true
Moves: [RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, RIGHT]
Total Cost: 87
Iterations: 15
Time: 1 ms

Iteration Log
Itteration: 1, Current Tile: (5, 3), Cost: 7, Path[RIGHT]
Itteration: 2, Current Tile: (3, 3), Cost: 17, Path[RIGHT, UP]
Itteration: 3, Current Tile: (3, 1), Cost: 27, Path[RIGHT, UP, LEFT]
Itteration: 3, Current Tile: (3, 5), Cost: 26, Path[RIGHT, UP, RIGHT]
Itteration: 4, Current Tile: (1, 5), Cost: 36, Path[RIGHT, UP, RIGHT, UP]
Itteration: 4, Current Tile: (3, 1), Cost: 44, Path[RIGHT, UP, RIGHT, LEFT]
Itteration: 5, Current Tile: (1, 1), Cost: 37, Path[RIGHT, UP, LEFT, UP]
Itteration: 6, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, RIGHT, UP, LEFT]
Itteration: 7, Current Tile: (5, 1), Cost: 61, Path[RIGHT, UP, LEFT, UP, DOWN]
Itteration: 7, Current Tile: (1, 5), Cost: 53, Path[RIGHT, UP, LEFT, UP, RIGHT]
Itteration: 11, Current Tile: (1, 1), Cost: 79, Path[RIGHT, UP, LEFT, UP, DOWN, UP]
Itteration: 11, Current Tile: (5, 3), Cost: 68, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT]
Itteration: 12, Current Tile: (3, 3), Cost: 78, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP]
Itteration: 13, Current Tile: (3, 1), Cost: 88, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, LEFT]
Itteration: 13, Current Tile: (3, 5), Cost: 87, Path[RIGHT, UP, LEFT, UP, DOWN, RIGHT, UP, RIGHT]

Itteration: 14, Current Tile: (1, 5), Cost: 95, Path[RIGHT, UP, LEFT, UP, DOWN, UP, RIGHT]

Iteration
Iteration: 1
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 2
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 3
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iteration: 4
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Iteration: 5
XXXXXXX
X0****X

X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 6
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 7
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 8
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iteration: 9
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 10

XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 11
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Iteration: 12
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iteration: 13
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

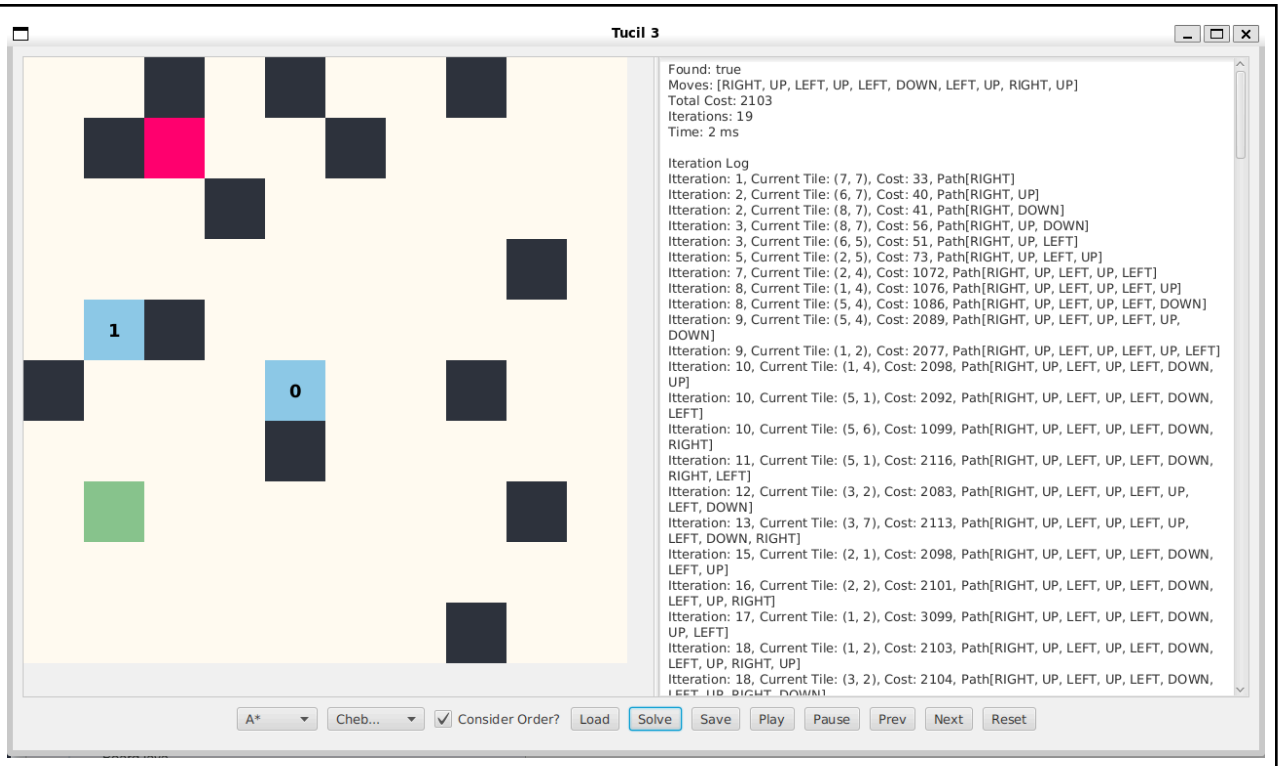
Iteration: 14
XXXXXXX
XZ****X
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iteration: 15
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Test Case 2

10 10
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X*****
*Z******X*
****
*******X**
1 2 999 4 5 6 7 999 8 9
1 999 2 4 999 999 5 6 7 8
1 2 3 999 1 5 6 7 8 9
1 2 3 4 5 6 7 8 999 0
1 2 999 3 4 5 6 7 8 9
1 999 3 4 5 6 7 999 8 9
1 2 3 4 999 5 6 7 8 9
1 2 3 4 5 6 7 8 999 9
1 2 3 4 5 6 7 8 9 0
1 2 3 4 5 6 7 999 9 0

Hasil



GBFS - Manhattan

Found: true
Moves: [RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, UP]
Total Cost: 1105
Iterations: 17
Time: 1 ms

Iteration Log
Itteration: 1, Current Tile: (7, 7), Cost: 33, Path[RIGHT]
Itteration: 2, Current Tile: (6, 7), Cost: 40, Path[RIGHT, UP]
Itteration: 2, Current Tile: (8, 7), Cost: 41, Path[RIGHT, DOWN]
Itteration: 3, Current Tile: (8, 7), Cost: 56, Path[RIGHT, UP, DOWN]
Itteration: 3, Current Tile: (6, 5), Cost: 51, Path[RIGHT, UP, LEFT]
Itteration: 4, Current Tile: (2, 5), Cost: 73, Path[RIGHT, UP, LEFT, UP]
Itteration: 5, Current Tile: (2, 4), Cost: 74, Path[RIGHT, UP, LEFT, UP, LEFT]
Itteration: 6, Current Tile: (1, 4), Cost: 1073, Path[RIGHT, UP, LEFT, UP, LEFT, UP]
Itteration: 6, Current Tile: (5, 4), Cost: 88, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN]
Itteration: 7, Current Tile: (5, 4), Cost: 1088, Path[RIGHT, UP, LEFT, UP, LEFT, UP, DOWN]
Itteration: 7, Current Tile: (1, 2), Cost: 1079, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT]
Itteration: 8, Current Tile: (3, 2), Cost: 1085, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT, DOWN]
Itteration: 9, Current Tile: (3, 7), Cost: 1115, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT, DOWN, RIGHT]

Itteration: 10, Current Tile: (1, 4), Cost: 1097, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP]
Itteration: 10, Current Tile: (5, 1), Cost: 1094, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT]
Itteration: 10, Current Tile: (5, 6), Cost: 101, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, RIGHT]
Itteration: 11, Current Tile: (1, 2), Cost: 1103, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP, LEFT]
Itteration: 12, Current Tile: (3, 2), Cost: 1109, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP, LEFT, DOWN]
Itteration: 13, Current Tile: (3, 7), Cost: 1139, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP, LEFT, DOWN, RIGHT]
Itteration: 14, Current Tile: (2, 1), Cost: 1100, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP]
Itteration: 14, Current Tile: (5, 6), Cost: 1119, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, RIGHT]
Itteration: 15, Current Tile: (2, 2), Cost: 1103, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT]
Itteration: 16, Current Tile: (1, 2), Cost: 1105, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, UP]
Itteration: 16, Current Tile: (3, 2), Cost: 1106, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, DOWN]

Iteration
Iteration: 1
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X*****
*Z******X*
****
*******X**

Iteration: 2
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X*****
*******ZX*
****
*******X**

Iteration: 3
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X**Z**
********X*
****
*******X**

Iteration: 4
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****XZ****
********X*
****
*******X**

Iteration: 5
X*XX**
*XO**X****
X*Z*
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

Iteration: 6
X*XX**
*XO**X****
XZ**
********X*
*1X*******

```
X***0**X**
****X*****
********X*
*********
*******X**
```

Iteration: 7

```
**X*X**X**
*XO*ZX****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*********
*******X**
```

Iteration: 8

```
**X*X**X**
*XZ**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*********
*******X**
```

Iteration: 9

```
**X*X**X**
*XO**X****
***X******
**Z*****X*
*1X*******
X***0**X**
****X*****
********X*
*********
*******X**
```

Iteration: 10

```
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***Z**X**
****X*****
********X*
**********
*******X**
```

Iteration: 11

```
**X*X**X**
*XO*ZX****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**
```

Iteration: 12

```
**X*X**X**
*XZ**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**
```

Iteration: 13

```
**X*X**X**
*XO**X****
***X******
**Z*****X*
*1X*******
X***0**X**
****X*****
```

```
********X*
**********
*******X**

Iteration: 14
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
XZ**0**X**
****X*****
********X*
**********
*******X**

Iteration: 15
**X*X**X**
*XO**X****
*Z*X******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**

Iteration: 16
**X*X**X**
*XO**X****
**ZX******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**

Iteration: 17
**X*X**X**
*XZ**X****
```

X***
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

UCS

Found: true
Moves: [RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, UP]
Total Cost: 1105
Iterations: 19
Time: 1 ms

Iteration Log
Itteration: 1, Current Tile: (7, 7), Cost: 33, Path[RIGHT]
Itteration: 2, Current Tile: (6, 7), Cost: 40, Path[RIGHT, UP]
Itteration: 2, Current Tile: (8, 7), Cost: 41, Path[RIGHT, DOWN]
Itteration: 3, Current Tile: (8, 7), Cost: 56, Path[RIGHT, UP, DOWN]
Itteration: 3, Current Tile: (6, 5), Cost: 51, Path[RIGHT, UP, LEFT]
Itteration: 5, Current Tile: (2, 5), Cost: 73, Path[RIGHT, UP, LEFT, UP]
Itteration: 7, Current Tile: (2, 4), Cost: 74, Path[RIGHT, UP, LEFT, UP, LEFT]
Itteration: 8, Current Tile: (1, 4), Cost: 1073, Path[RIGHT, UP, LEFT, UP, LEFT, UP]
Itteration: 8, Current Tile: (5, 4), Cost: 88, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN]
Itteration: 9, Current Tile: (1, 4), Cost: 1097, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP]
Itteration: 9, Current Tile: (5, 1), Cost: 1094, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT]
Itteration: 9, Current Tile: (5, 6), Cost: 101, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, RIGHT]
Itteration: 10, Current Tile: (5, 1), Cost: 1118, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, RIGHT, LEFT]
Itteration: 11, Current Tile: (1, 2), Cost: 1079, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT]
Itteration: 12, Current Tile: (3, 2), Cost: 1085, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT, DOWN]
Itteration: 13, Current Tile: (3, 7), Cost: 1115, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT, DOWN, RIGHT]
Itteration: 14, Current Tile: (2, 1), Cost: 1100, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP]
Itteration: 15, Current Tile: (1, 2), Cost: 1103, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP, LEFT]
Itteration: 16, Current Tile: (2, 2), Cost: 1103, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT]
Itteration: 17, Current Tile: (3, 2), Cost: 1109, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP, LEFT, DOWN]
Itteration: 18, Current Tile: (1, 2), Cost: 1105, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, UP]

Itteration: 18, Current Tile: (3, 2), Cost: 1106, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, DOWN]

Iteration
Iteration: 1
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X*****
*Z******X*
****
*******X**

Iteration: 2
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X*****
*******ZX*
****
*******X**

Iteration: 3
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X**Z**
********X*
****
*******X**

Iteration: 4
X*XX**

```
*XO**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*******Z**
*******X**

Iteration: 5
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***0**X**
****XZ****
********X*
**********
*******X**

Iteration: 6
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*******Z**
*******X**

Iteration: 7
**X*X**X**
*XO**X****
***X*Z****
********X*
*1X*******
X***0**X**
****X*****
********X*
```

```
*********
*******X**

Iteration: 8
**X*X**X**
*XO**X****
***XZ*****
********X*
*1X*******
X***0**X**
****X*****
********X*
*********
*******X**

Iteration: 9
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***Z**X**
****X*****
********X*
*********
*******X**

Iteration: 10
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***0*ZX**
****X*****
********X*
*********
*******X**

Iteration: 11
**X*X**X**
*XO*ZX****
***X******
```

```
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**
```

Iteration: 12

```
**X*X**X**
*XZ**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**
```

Iteration: 13

```
**X*X**X**
*XO**X****
***X******
**Z*****X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**
```

Iteration: 14

```
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
XZ**0**X**
****X*****
********X*
**********
*******X**
```

Iteration: 15
X*XX**
*XO*ZX****
X***
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

Iteration: 16
X*XX**
*XO**X****
*Z*X******
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

Iteration: 17
X*XX**
*XZ**X****
X***
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

Iteration: 18
X*XX**
*XO**X****
ZX****
********X*
*1X*******

X***0**X**
****X*****
********X*
****
*******X**

Iteration: 19
X*XX**
*XZ**X****
X***
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

A* - Manhattan

Found: true
Moves: [RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, UP]
Total Cost: 1105
Iterations: 19
Time: 2 ms

Iteration Log
Itteration: 1, Current Tile: (7, 7), Cost: 33, Path[RIGHT]
Itteration: 2, Current Tile: (6, 7), Cost: 40, Path[RIGHT, UP]
Itteration: 2, Current Tile: (8, 7), Cost: 41, Path[RIGHT, DOWN]
Itteration: 3, Current Tile: (8, 7), Cost: 56, Path[RIGHT, UP, DOWN]
Itteration: 3, Current Tile: (6, 5), Cost: 51, Path[RIGHT, UP, LEFT]
Itteration: 5, Current Tile: (2, 5), Cost: 73, Path[RIGHT, UP, LEFT, UP]
Itteration: 7, Current Tile: (2, 4), Cost: 74, Path[RIGHT, UP, LEFT, UP, LEFT]
Itteration: 8, Current Tile: (1, 4), Cost: 1073, Path[RIGHT, UP, LEFT, UP, LEFT, UP]
Itteration: 8, Current Tile: (5, 4), Cost: 88, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN]
Itteration: 9, Current Tile: (1, 4), Cost: 1097, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP]
Itteration: 9, Current Tile: (5, 1), Cost: 1094, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT]
Itteration: 9, Current Tile: (5, 6), Cost: 101, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, RIGHT]
Itteration: 10, Current Tile: (5, 1), Cost: 1118, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, RIGHT, LEFT]
Itteration: 11, Current Tile: (1, 2), Cost: 1079, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT]
Itteration: 12, Current Tile: (3, 2), Cost: 1085, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT, DOWN]
Itteration: 13, Current Tile: (3, 7), Cost: 1115, Path[RIGHT, UP, LEFT, UP, LEFT, UP, LEFT, DOWN,

RIGHT]
Itteration: 14, Current Tile: (1, 2), Cost: 1103, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP, LEFT]
Itteration: 15, Current Tile: (2, 1), Cost: 1100, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP]
Itteration: 16, Current Tile: (2, 2), Cost: 1103, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT]
Itteration: 17, Current Tile: (3, 2), Cost: 1109, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, UP, LEFT, DOWN]
Itteration: 18, Current Tile: (1, 2), Cost: 1105, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, UP]
Itteration: 18, Current Tile: (3, 2), Cost: 1106, Path[RIGHT, UP, LEFT, UP, LEFT, DOWN, LEFT, UP, RIGHT, DOWN]

Iteration
Iteration: 1
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X*****
*Z******X*
****
*******X**

Iteration: 2
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0**X**
****X*****
*******ZX*
****
*******X**

Iteration: 3
X*XX**
*XO**X****
X***
********X*

```
*1X*******
X***0**X**
****X**Z**
********X*
**********
*******X**
```

Iteration: 4

```
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*******Z**
*******X**
```

Iteration: 5

```
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***0**X**
****XZ****
********X*
**********
*******X**
```

Iteration: 6

```
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*******Z**
*******X**
```

Iteration: 7
X*XX**
*XO**X****
X*Z*
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

Iteration: 8
X*XX**
*XO**X****
XZ**
********X*
*1X*******
X***0**X**
****X*****
********X*
****
*******X**

Iteration: 9
X*XX**
*XO**X****
X***
********X*
*1X*******
X***Z**X**
****X*****
********X*
****
*******X**

Iteration: 10
X*XX**
*XO**X****
X***
********X*
*1X*******
X***0*ZX**

```
****X*****
********X*
*********
*******X**

Iteration: 11
**X*X**X**
*XO*ZX****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*********
*******X**

Iteration: 12
**X*X**X**
*XZ**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
*********
*******X**

Iteration: 13
**X*X**X**
*XO**X****
***X******
**Z*****X*
*1X*******
X***0**X**
****X*****
********X*
*********
*******X**

Iteration: 14
**X*X**X**
```

```
*XO*ZX****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**

Iteration: 15
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
XZ**0**X**
****X*****
********X*
**********
*******X**

Iteration: 16
**X*X**X**
*XO**X****
*Z*X******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**

Iteration: 17
**X*X**X**
*XZ**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
```

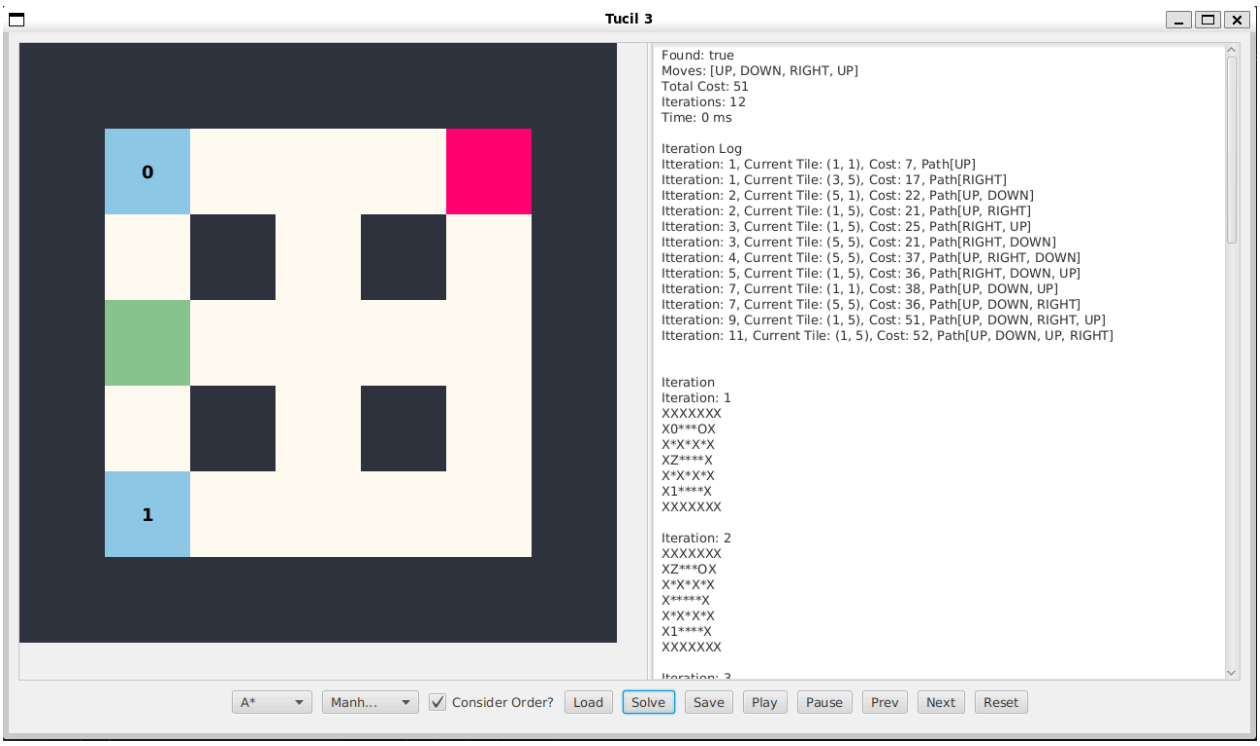
```
**********
*******X**

Iteration: 18
**X*X**X**
*XO**X****
**ZX******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**

Iteration: 19
**X*X**X**
*XZ**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
********X*
**********
*******X**
```

Test Case 3

```
7 7
XXXXXXX
X0***OX
X*X*X*X
XZ****X
X*X*X*X
X1****X
XXXXXXX
999 999 999 999 999 999 999
999 2 4 6 3 1 999
999 5 999 8 999 7 999
999 3 2 4 6 5 999
999 6 999 7 999 2 999
```

999 1 3 5 4 2 999
999 999 999 999 999 999 999

Hasil



A* - Manhattan

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 51
Iterations: 12
Time: 0 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 7, Path[UP]
Itteration: 1, Current Tile: (3, 5), Cost: 17, Path[RIGHT]
Itteration: 2, Current Tile: (5, 1), Cost: 22, Path[UP, DOWN]
Itteration: 2, Current Tile: (1, 5), Cost: 21, Path[UP, RIGHT]
Itteration: 3, Current Tile: (1, 5), Cost: 25, Path[RIGHT, UP]
Itteration: 3, Current Tile: (5, 5), Cost: 21, Path[RIGHT, DOWN]
Itteration: 4, Current Tile: (5, 5), Cost: 37, Path[UP, RIGHT, DOWN]
Itteration: 5, Current Tile: (1, 5), Cost: 36, Path[RIGHT, DOWN, UP]
Itteration: 7, Current Tile: (1, 1), Cost: 38, Path[UP, DOWN, UP]
Itteration: 7, Current Tile: (5, 5), Cost: 36, Path[UP, DOWN, RIGHT]

Itteration: 9, Current Tile: (1, 5), Cost: 51, Path[UP, DOWN, RIGHT, UP]
Itteration: 11, Current Tile: (1, 5), Cost: 52, Path[UP, DOWN, UP, RIGHT]

Iteration
Iteration: 1
XXXXXXX
X0***OX
X*X*X*X
XZ****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 2
XXXXXXX
XZ***OX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 3
XXXXXXX
X0***OX
X*X*X*X
X****ZX
X*X*X*X
X1****X
XXXXXXX

Iteration: 4
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 5
XXXXXXX

X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX
XXXXXXX

Iteration: 6
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 7
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
XZ****X
XXXXXXX

Iteration: 8
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 9
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX
XXXXXXX

Iteration: 10
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX
XXXXXXX

Iteration: 11
XXXXXXX
XZ***OX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 12
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

GBFS - Manhattan

Found: true
Moves: [RIGHT, UP, LEFT, RIGHT, DOWN, LEFT, UP, RIGHT]
Total Cost: 113
Iterations: 12
Time: 1 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 7, Path[UP]
Itteration: 1, Current Tile: (3, 5), Cost: 17, Path[RIGHT]
Itteration: 2, Current Tile: (1, 5), Cost: 25, Path[RIGHT, UP]
Itteration: 2, Current Tile: (5, 5), Cost: 21, Path[RIGHT, DOWN]
Itteration: 3, Current Tile: (5, 5), Cost: 41, Path[RIGHT, UP, DOWN]
Itteration: 3, Current Tile: (1, 1), Cost: 40, Path[RIGHT, UP, LEFT]
Itteration: 5, Current Tile: (5, 1), Cost: 55, Path[RIGHT, UP, LEFT, DOWN]
Itteration: 5, Current Tile: (1, 5), Cost: 54, Path[RIGHT, UP, LEFT, RIGHT]

Itteration: 6, Current Tile: (5, 5), Cost: 70, Path[RIGHT, UP, LEFT, RIGHT, DOWN]
Itteration: 8, Current Tile: (5, 1), Cost: 83, Path[RIGHT, UP, LEFT, RIGHT, DOWN, LEFT]
Itteration: 10, Current Tile: (1, 1), Cost: 99, Path[RIGHT, UP, LEFT, RIGHT, DOWN, LEFT, UP]
Itteration: 10, Current Tile: (5, 5), Cost: 97, Path[RIGHT, UP, LEFT, RIGHT, DOWN, LEFT, RIGHT]
Itteration: 11, Current Tile: (1, 5), Cost: 113, Path[RIGHT, UP, LEFT, RIGHT, DOWN, LEFT, UP, RIGHT]

Iteration
Iteration: 1
XXXXXXX
X0***OX
X*X*X*X
XZ****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 2
XXXXXXX
X0***OX
X*X*X*X
X****ZX
X*X*X*X
X1****X
XXXXXXX

Iteration: 3
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 4
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX

XXXXXXX

Iteration: 5
XXXXXXX
XZ***OX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 6
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 7
XXXXXXX
XZ***OX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 8
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX
XXXXXXX

Iteration: 9
XXXXXXX
X0***OX
X*X*X*X
X*****X

X*X*X*X
X1***ZX
XXXXXXX

Iteration: 10
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
XZ****X
XXXXXXX

Iteration: 11
XXXXXXX
XZ***OX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 12
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

UCS

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 51
Iterations: 12
Time: 0 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 7, Path[UP]
Itteration: 1, Current Tile: (3, 5), Cost: 17, Path[RIGHT]
Itteration: 2, Current Tile: (5, 1), Cost: 22, Path[UP, DOWN]
Itteration: 2, Current Tile: (1, 5), Cost: 21, Path[UP, RIGHT]

Itteration: 3, Current Tile: (1, 5), Cost: 25, Path[RIGHT, UP]
Itteration: 3, Current Tile: (5, 5), Cost: 21, Path[RIGHT, DOWN]
Itteration: 4, Current Tile: (5, 5), Cost: 37, Path[UP, RIGHT, DOWN]
Itteration: 5, Current Tile: (1, 5), Cost: 36, Path[RIGHT, DOWN, UP]
Itteration: 6, Current Tile: (1, 1), Cost: 38, Path[UP, DOWN, UP]
Itteration: 6, Current Tile: (5, 5), Cost: 36, Path[UP, DOWN, RIGHT]
Itteration: 8, Current Tile: (1, 5), Cost: 51, Path[UP, DOWN, RIGHT, UP]
Itteration: 11, Current Tile: (1, 5), Cost: 52, Path[UP, DOWN, UP, RIGHT]

Iteration
Iteration: 1
XXXXXXX
X0***OX
X*X*X*X
XZ****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 2
XXXXXXX
XZ***OX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 3
XXXXXXX
X0***OX
X*X*X*X
X****ZX
X*X*X*X
X1****X
XXXXXXX

Iteration: 4
XXXXXXX
X0***ZX
X*X*X*X
X*****X

X*X*X*X
X1****X
XXXXXXX

Iteration: 5
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX
XXXXXXX

Iteration: 6
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
XZ****X
XXXXXXX

Iteration: 7
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 8
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX
XXXXXXX

Iteration: 9
XXXXXXX
X0***ZX

```
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 10
XXXXXXX
X0***OX
X*X*X*X
X*****X
X*X*X*X
X1***ZX
XXXXXXX

Iteration: 11
XXXXXXX
XZ***OX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX

Iteration: 12
XXXXXXX
X0***ZX
X*X*X*X
X*****X
X*X*X*X
X1****X
XXXXXXX
```

Test Case 4
7 7 XXXXXXX X0*L*OX X*X*X*X XZ****X X*XXX*X X1****X

XXXXXXX
999 999 999 999 999 999 999
999 3 4 999 6 2 999
999 7 999 5 999 4 999
999 2 3 6 5 1 999
999 8 999 999 999 7 999
999 1 2 4 3 2 999
999 999 999 999 999 999 999

Hasil

UCS

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 53
Iterations: 10
Time: 1 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 10, Path[UP]
Itteration: 1, Current Tile: (3, 5), Cost: 15, Path[RIGHT]
Itteration: 2, Current Tile: (5, 1), Cost: 28, Path[UP, DOWN]
Itteration: 3, Current Tile: (1, 5), Cost: 21, Path[RIGHT, UP]
Itteration: 3, Current Tile: (5, 5), Cost: 24, Path[RIGHT, DOWN]
Itteration: 4, Current Tile: (5, 5), Cost: 35, Path[RIGHT, UP, DOWN]
Itteration: 6, Current Tile: (1, 1), Cost: 48, Path[UP, DOWN, UP]
Itteration: 6, Current Tile: (5, 5), Cost: 39, Path[UP, DOWN, RIGHT]
Itteration: 8, Current Tile: (1, 5), Cost: 53, Path[UP, DOWN, RIGHT, UP]

Iteration
Iteration: 1
XXXXXXX
X0*L*OX
X*X*X*X
XZ****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 2

```
XXXXXXX
XZ*L*OX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 3
XXXXXXX
X0*L*OX
X*X*X*X
X****ZX
X*XXX*X
X1****X
XXXXXXX

Iteration: 4
XXXXXXX
X0*L*ZX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 5
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 6
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
XZ****X
XXXXXXX
```

Iteration: 7
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 8
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 9
XXXXXXX
XZ*L*OX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 10
XXXXXXX
X0*L*ZX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

GBFS - Euclidian

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 53
Iterations: 10
Time: 0 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 10, Path[UP]
Itteration: 1, Current Tile: (3, 5), Cost: 15, Path[RIGHT]
Itteration: 2, Current Tile: (1, 5), Cost: 21, Path[RIGHT, UP]
Itteration: 2, Current Tile: (5, 5), Cost: 24, Path[RIGHT, DOWN]
Itteration: 3, Current Tile: (5, 5), Cost: 35, Path[RIGHT, UP, DOWN]
Itteration: 6, Current Tile: (5, 1), Cost: 28, Path[UP, DOWN]
Itteration: 7, Current Tile: (1, 1), Cost: 48, Path[UP, DOWN, UP]
Itteration: 7, Current Tile: (5, 5), Cost: 39, Path[UP, DOWN, RIGHT]
Itteration: 9, Current Tile: (1, 5), Cost: 53, Path[UP, DOWN, RIGHT, UP]

Iteration
Iteration: 1
XXXXXXX
X0*L*OX
X*X*X*X
XZ****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 2
XXXXXXX
X0*L*OX
X*X*X*X
X****ZX
X*XXX*X
X1****X
XXXXXXX

Iteration: 3
XXXXXXX
X0*L*ZX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 4
XXXXXXX

X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 5
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 6
XXXXXXX
XZ*L*OX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 7
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
XZ****X
XXXXXXX

Iteration: 8
XXXXXXX
XZ*L*OX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 9
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 10
XXXXXXX
X0*L*ZX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

A* - Chebyshev

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 53
Iterations: 10
Time: 0 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 10, Path[UP]
Itteration: 1, Current Tile: (3, 5), Cost: 15, Path[RIGHT]
Itteration: 2, Current Tile: (5, 1), Cost: 28, Path[UP, DOWN]
Itteration: 3, Current Tile: (1, 5), Cost: 21, Path[RIGHT, UP]
Itteration: 3, Current Tile: (5, 5), Cost: 24, Path[RIGHT, DOWN]
Itteration: 4, Current Tile: (5, 5), Cost: 35, Path[RIGHT, UP, DOWN]
Itteration: 6, Current Tile: (1, 1), Cost: 48, Path[UP, DOWN, UP]
Itteration: 6, Current Tile: (5, 5), Cost: 39, Path[UP, DOWN, RIGHT]
Itteration: 8, Current Tile: (1, 5), Cost: 53, Path[UP, DOWN, RIGHT, UP]

Iteration
Iteration: 1
XXXXXXX
X0*L*OX
X*X*X*X
XZ****X

X*XXX*X
X1****X
XXXXXXX

Iteration: 2
XXXXXXX
XZ*L*OX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 3
XXXXXXX
X0*L*OX
X*X*X*X
X****ZX
X*XXX*X
X1****X
XXXXXXX

Iteration: 4
XXXXXXX
X0*L*ZX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 5
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 6
XXXXXXX
X0*L*OX

X*X*X*X
X*****X
X*XXX*X
XZ****X
XXXXXXX

Iteration: 7
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

Iteration: 8
XXXXXXX
X0*L*OX
X*X*X*X
X*****X
X*XXX*X
X1***ZX
XXXXXXX

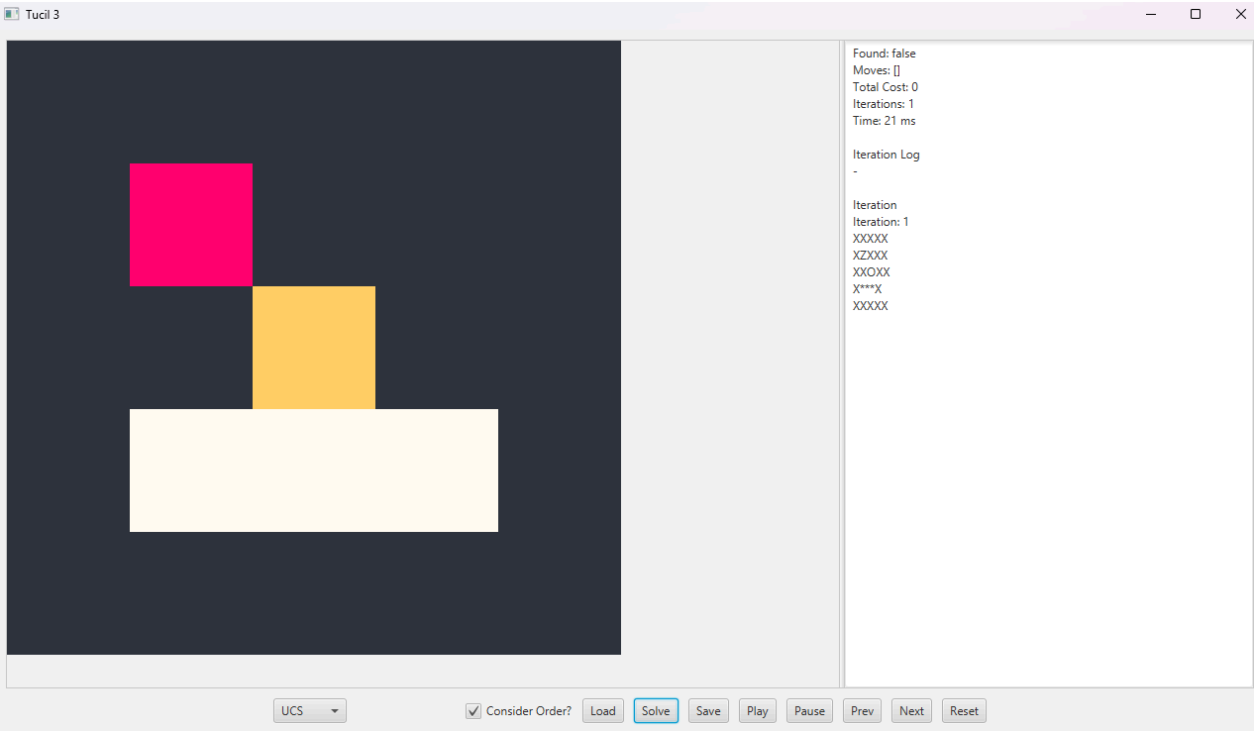
Iteration: 9
XXXXXXX
XZ*L*OX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Iteration: 10
XXXXXXX
X0*L*ZX
X*X*X*X
X*****X
X*XXX*X
X1****X
XXXXXXX

Test Case 5

5 5
XXXXX
XZXXX
XXOXX
X***X
XXXXX
999 999 999 999 999
999 2 999 999 999
999 999 5 999 999
999 3 4 6 999
999 999 999 999 999

Hasil



UCS

Found: false
Moves: []
Total Cost: 0
Iterations: 1
Time: 21 ms

Iteration Log
-

Iteration
Iteration: 1
XXXXX
XZXXX
XXOXX
X***X
XXXXX

GBFS - Manhattan

Found: false
Moves: []
Total Cost: 0
Iterations: 1
Time: 0 ms

Iteration Log
-

Iteration
Iteration: 1
XXXXX
XZXXX
XXOXX
X***X
XXXXX

GBFS - Euclidean

Found: false
Moves: []
Total Cost: 0
Iterations: 1
Time: 0 ms

Iteration Log
-

Iteration
Iteration: 1

XXXXX
XZXXX
XXOXX
X***X
XXXXX

GBFS - Chebysev

Found: false
Moves: []
Total Cost: 0
Iterations: 1
Time: 0 ms

Iteration Log
-

Iteration
Iteration: 1
XXXXX
XZXXX
XXOXX
X***X
XXXXX

A*- Manhattan

Found: false
Moves: []
Total Cost: 0
Iterations: 1
Time: 2 ms

Iteration Log
-

Iteration
Iteration: 1
XXXXX
XZXXX
XXOXX
X***X
XXXXX

A*- Euclidean

Found: false
Moves: []
Total Cost: 0
Iterations: 1
Time: 0 ms

Iteration Log
-

Iteration
Iteration: 1
XXXXX
XZXXX
XXOXX
X***X
XXXXX

A*- Chebysev

Found: false
Moves: []
Total Cost: 0
Iterations: 1
Time: 0 ms

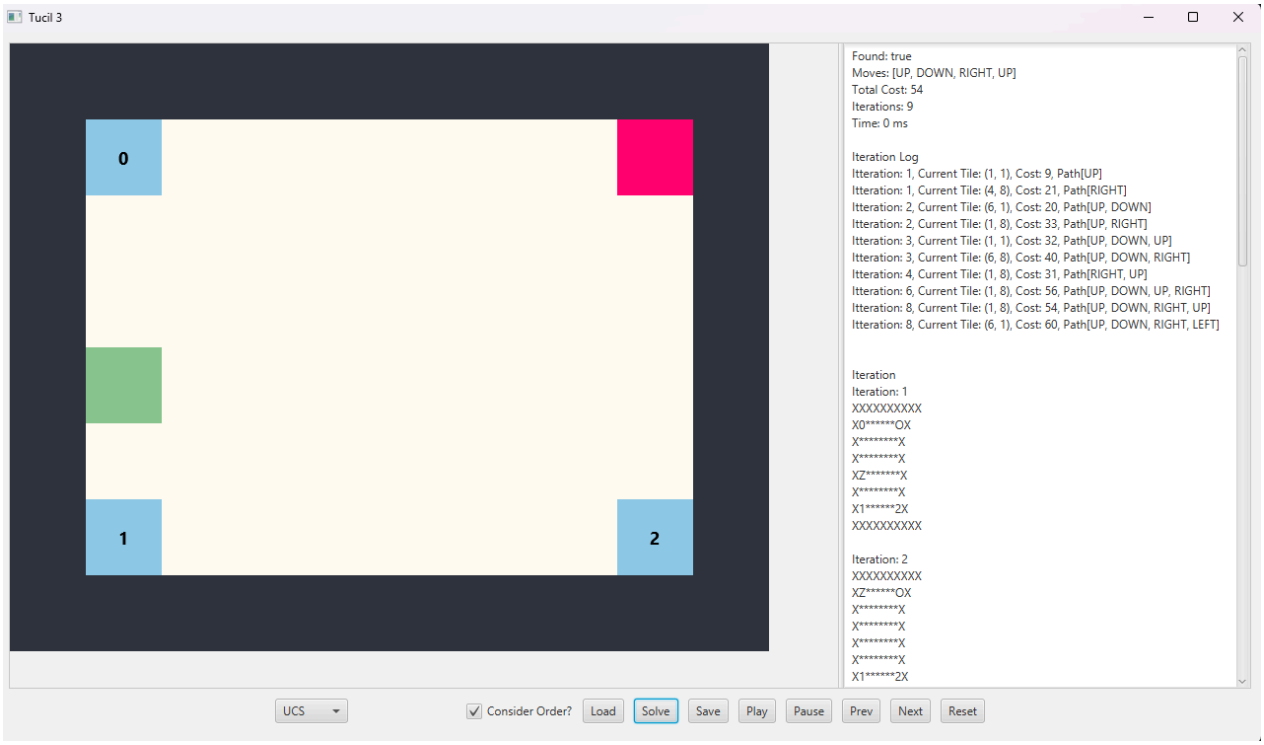
Iteration Log
-

Iteration
Iteration: 1
XXXXX
XZXXX
XXOXX
X***X
XXXXX

Test Case 6

8 10
XXXXXXXXXX

X0******OX
X********X
X********X
XZ*******X
X********X
X1******2X
XXXXXXXXXX
999 999 999 999 999 999 999 999 999 999
999 2 3 4 1 5 6 2 3 999
999 4 1 2 3 5 2 4 1 999
999 3 2 5 4 1 3 2 6 999
999 1 4 2 5 3 2 4 1 999
999 2 3 1 4 6 2 5 3 999
999 1 2 3 4 5 2 3 1 999
999 999 999 999 999 999 999 999 999 999



UCS

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 54
Iterations: 9
Time: 0 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 9, Path[UP]
Itteration: 1, Current Tile: (4, 8), Cost: 21, Path[RIGHT]
Itteration: 2, Current Tile: (6, 1), Cost: 20, Path[UP, DOWN]
Itteration: 2, Current Tile: (1, 8), Cost: 33, Path[UP, RIGHT]
Itteration: 3, Current Tile: (1, 1), Cost: 32, Path[UP, DOWN, UP]
Itteration: 3, Current Tile: (6, 8), Cost: 40, Path[UP, DOWN, RIGHT]
Itteration: 4, Current Tile: (1, 8), Cost: 31, Path[RIGHT, UP]
Itteration: 6, Current Tile: (1, 8), Cost: 56, Path[UP, DOWN, UP, RIGHT]
Itteration: 8, Current Tile: (1, 8), Cost: 54, Path[UP, DOWN, RIGHT, UP]
Itteration: 8, Current Tile: (6, 1), Cost: 60, Path[UP, DOWN, RIGHT, LEFT]

Iteration
Iteration: 1
XXXXXXXXXX
X0******OX
X********X
X********X
XZ*******X
X********X
X1******2X
XXXXXXXXXX

Iteration: 2
XXXXXXXXXX
XZ******OX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 3
XXXXXXXXXX
X0******OX
X********X
X********X
X********X
X********X
XZ******2X
XXXXXXXXXX

GBFS - Manhattan

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 54
Iterations: 9
Time: 0 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 9, Path[UP]
Itteration: 1, Current Tile: (4, 8), Cost: 21, Path[RIGHT]
Itteration: 2, Current Tile: (1, 8), Cost: 31, Path[RIGHT, UP]
Itteration: 3, Current Tile: (1, 1), Cost: 54, Path[RIGHT, UP, LEFT]
Itteration: 4, Current Tile: (6, 1), Cost: 20, Path[UP, DOWN]
Itteration: 4, Current Tile: (1, 8), Cost: 33, Path[UP, RIGHT]
Itteration: 7, Current Tile: (1, 1), Cost: 32, Path[UP, DOWN, UP]
Itteration: 7, Current Tile: (6, 8), Cost: 40, Path[UP, DOWN, RIGHT]
Itteration: 8, Current Tile: (1, 8), Cost: 54, Path[UP, DOWN, RIGHT, UP]
Itteration: 8, Current Tile: (6, 1), Cost: 60, Path[UP, DOWN, RIGHT, LEFT]

Iteration
Iteration: 1
XXXXXXXXXX
X0******OX
X********X
X********X
XZ*******X
X********X
X1******2X
XXXXXXXXXX

Iteration: 2
XXXXXXXXXX
X0******OX
X********X
X********X
X*******ZX
X********X
X1******2X
XXXXXXXXXX

Iteration: 3

XXXXXXXXXX
X0******ZX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 4
XXXXXXXXXX
XZ******OX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 5
XXXXXXXXXX
X0******ZX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 6
XXXXXXXXXX
XZ******OX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 7
XXXXXXXXXX
X0******OX
X********X

X********X
X********X
X********X
XZ******2X
XXXXXXXXXX

Iteration: 8
XXXXXXXXXX
X0******OX
X********X
X********X
X********X
X********X
X1******ZX
XXXXXXXXXX

Iteration: 9
XXXXXXXXXX
X0******ZX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

A* - Euclidean

Found: true
Moves: [UP, DOWN, RIGHT, UP]
Total Cost: 54
Iterations: 9
Time: 1 ms

Iteration Log
Itteration: 1, Current Tile: (1, 1), Cost: 9, Path[UP]
Itteration: 1, Current Tile: (4, 8), Cost: 21, Path[RIGHT]
Itteration: 2, Current Tile: (6, 1), Cost: 20, Path[UP, DOWN]
Itteration: 2, Current Tile: (1, 8), Cost: 33, Path[UP, RIGHT]
Itteration: 3, Current Tile: (1, 8), Cost: 31, Path[RIGHT, UP]
Itteration: 4, Current Tile: (1, 1), Cost: 32, Path[UP, DOWN, UP]
Itteration: 4, Current Tile: (6, 8), Cost: 40, Path[UP, DOWN, RIGHT]
Itteration: 7, Current Tile: (1, 8), Cost: 56, Path[UP, DOWN, UP, RIGHT]
Itteration: 8, Current Tile: (1, 8), Cost: 54, Path[UP, DOWN, RIGHT, UP]

Itteration: 8, Current Tile: (6, 1), Cost: 60, Path[UP, DOWN, RIGHT, LEFT]

Iteration
Iteration: 1
XXXXXXXXXX
X0******OX
X********X
X********X
XZ*******X
X********X
X1******2X
XXXXXXXXXX

Iteration: 2
XXXXXXXXXX
XZ******OX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 3
XXXXXXXXXX
X0******OX
X********X
X********X
X*******ZX
X********X
X1******2X
XXXXXXXXXX

Iteration: 4
XXXXXXXXXX
X0******OX
X********X
X********X
X********X
X********X
XZ******2X
XXXXXXXXXX

Iteration: 5
XXXXXXXXXX
X0******ZX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 6
XXXXXXXXXX
X0******ZX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 7
XXXXXXXXXX
XZ******OX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX

Iteration: 8
XXXXXXXXXX
X0******OX
X********X
X********X
X********X
X********X
X1******ZX
XXXXXXXXXX

Iteration: 9
XXXXXXXXXX

```
X0******ZX
X********X
X********X
X********X
X********X
X1******2X
XXXXXXXXXX
```

BAB IV

ANALISIS HASIL

1. Analisis Hasil Test Case

   Pada Test Case 1, UCS dan A* menghasilkan solusi yang sama dengan Total Cost = 87, sedangkan GBFS menghasilkan solusi dengan Total Cost = 104. Selisih ini menunjukkan bahwa GBFS memang dapat menemukan goal lebih cepat secara arah, tetapi tidak selalu menempuh jalur termurah, sama hal nya dengan Test Case 3.

   Pada Test Case 2, semua algoritma yang diuji menghasilkan solusi yang sama dengan Total Cost = 1105. Walaupun jumlah iterasi UCS dan A* sedikit lebih besar daripada GBFS, perbedaan hasil akhirnya tidak ada, menunjukkan bahwa struktur papan dan constraint urutan tile dapat memberikan pengaruh yang cukup global ke solusi, solusi memiliki cost > 1000 karena ada tile dengan cost 999 yang passable.

   Pada Test Case 4, seluruh algoritma yang diuji menghasilkan solusi identik dengan Total Cost = 53 dan jumlah iterasi yang sama. Hal ini menunjukkan bahwa pada papan yang ruang geraknya lebih terbatas dan solusi validnya relatif jelas, perbedaan strategi prioritas node tidak terlalu berpengaruh. Keberadaan tile L pada kasus ini juga sudah tertangani dengan benar karena semua algoritma mampu menghindari jalur yang tidak valid.

   Pada Test Case 5, seluruh algoritma menghasilkan Found: false hanya dalam 1 iterasi. Menunjukkan bahwa implementasi mampu mengenali kasus tanpa solusi tanpa melakukan ekspansi lebih.

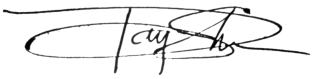
2. Kesimpulan

   Dari keseluruhan pengujian, UCS selalu memberikan cost minimum di antara algoritma yang diuji. A* menunjukkan performa yang seimbang, selalu menyamai kualitas solusi UCS pada test case yang ditampilkan, tetapi tetap memiliki pencarian yang lebih terarah karena menggunakan heuristic. Oleh karena itu, A* dapat dianggap sebagai pendekatan yang paling rasional untuk puzzle ini. GBFS merupakan algoritma yang paling agresif menuju goal. Sifat ini menguntungkan pada beberapa kasus sederhana, tetapi menjadi kelemahan pada kasus yang memiliki constraint ordered tile, atau beberapa lintasan dengan cost sangat berbeda.

Lampiran

Repository GitHub: https://github.com/daypft/Tucil3_13524050_13524133

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Raysha Erviandika Putra - 13524050

Muhammad Daffa Arrizki Yanma - 13524133

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma *pathfinding* alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	